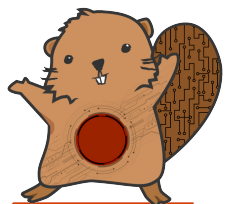




Machine Learning and Data Mining

Lecture 7.2: Advanced Neural Networks & Decision Trees





Element-wise Logistic with 3 Inputs:

Forward:

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \sigma \left(\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \right) = \begin{bmatrix} 1/(1 + e^{-z_1}) \\ 1/(1 + e^{-z_2}) \\ 1/(1 + e^{-z_3}) \end{bmatrix}$$

Backward:

$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \begin{bmatrix} \frac{\delta a_1}{\delta z_1} & \frac{\delta a_1}{\delta z_2} & \frac{\delta a_1}{\delta z_3} \\ \frac{\delta a_2}{\delta z_1} & \frac{\delta a_2}{\delta z_2} & \frac{\delta a_2}{\delta z_3} \\ \frac{\delta a_3}{\delta z_1} & \frac{\delta a_3}{\delta z_2} & \frac{\delta a_3}{\delta z_3} \end{bmatrix}$$

$$\frac{\delta a_i}{\delta z_i} = a_i(1 - a_i)$$

$$\frac{\delta a_i}{\delta z_j} = 0$$

$$\frac{\partial \mathbf{a}}{\partial \mathbf{z}} = \begin{bmatrix} a_1(1 - a_1) & 0 & 0 \\ 0 & a_2(1 - a_2) & 0 \\ 0 & 0 & a_3(1 - a_3) \end{bmatrix}$$



Linear layer with 5 inputs and 3 outputs:

Forward:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \sigma \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \right) = W\mathbf{x} + \mathbf{b} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Backward: (Output with respect to input)

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial z_1}{\partial x_1} & \frac{\partial z_1}{\partial x_2} & \frac{\partial z_1}{\partial x_3} & \frac{\partial z_1}{\partial x_4} & \frac{\partial z_1}{\partial x_5} \\ \frac{\partial z_2}{\partial x_1} & \frac{\partial z_2}{\partial x_2} & \frac{\partial z_2}{\partial x_3} & \frac{\partial z_2}{\partial x_4} & \frac{\partial z_2}{\partial x_5} \\ \frac{\partial z_3}{\partial x_1} & \frac{\partial z_3}{\partial x_2} & \frac{\partial z_3}{\partial x_3} & \frac{\partial z_3}{\partial x_4} & \frac{\partial z_3}{\partial x_5} \end{bmatrix} \quad \frac{\delta z_i}{\delta x_j} = w_{ij} \quad \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix} = W$$



Linear layer with 5 inputs and 3 outputs:

Forward:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \sigma \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \right) = W\mathbf{x} + \mathbf{b} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Backward: (Output with respect to bias)

$$\frac{\partial \mathbf{z}}{\partial \mathbf{b}} = \begin{bmatrix} \frac{\partial z_1}{\partial b_1} & \frac{\partial z_1}{\partial b_2} & \frac{\partial z_1}{\partial b_3} \\ \frac{\partial z_2}{\partial b_1} & \frac{\partial z_2}{\partial b_2} & \frac{\partial z_2}{\partial b_3} \\ \frac{\partial z_3}{\partial b_1} & \frac{\partial z_3}{\partial b_2} & \frac{\partial z_3}{\partial b_3} \end{bmatrix}$$

$$\frac{\delta z_i}{\delta b_i} = 1$$

$$\frac{\delta z_i}{\delta b_{j \neq i}} = 0$$

$$\frac{\partial \mathbf{z}}{\partial \mathbf{b}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I$$



Linear layer with 5 inputs and 3 outputs:

Forward:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \sigma \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \right) = W\mathbf{x} + \mathbf{b} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Backward: (Output with respect to weights)

$$\frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial z_1}{\partial w_{11}} & \frac{\partial z_1}{\partial w_{12}} & \dots & \frac{\partial z_1}{\partial w_{35}} \\ \frac{\partial z_2}{\partial w_{11}} & \frac{\partial z_2}{\partial w_{12}} & \dots & \frac{\partial z_2}{\partial w_{35}} \\ \frac{\partial z_3}{\partial w_{11}} & \frac{\partial z_3}{\partial w_{12}} & \dots & \frac{\partial z_3}{\partial w_{35}} \end{bmatrix}$$

$$\frac{\partial z_i}{\partial w_{ik}} = x_k$$

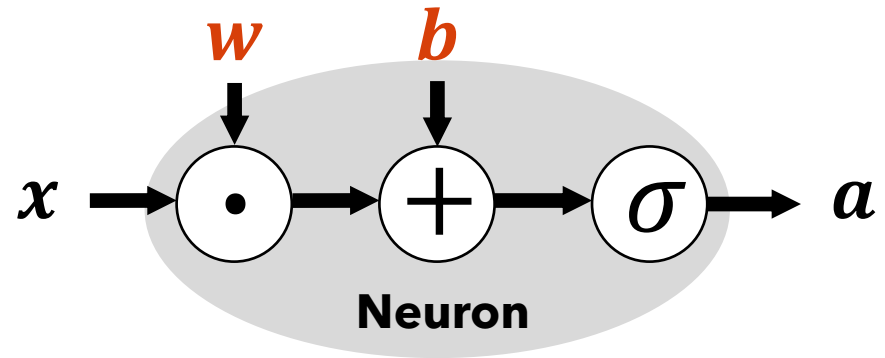
$$\frac{\partial z_i}{\partial w_{j \neq i, k}} = 0$$

$$\frac{\partial \mathbf{z}}{\partial \mathbf{W}} = \begin{bmatrix} x^T & 0 \dots 0 & 0 \dots 0 \\ 0 \dots 0 & x^T & 0 \dots 0 \\ 0 \dots 0 & 0 \dots 0 & x^T \end{bmatrix}$$



RECAP

From Last Lecture



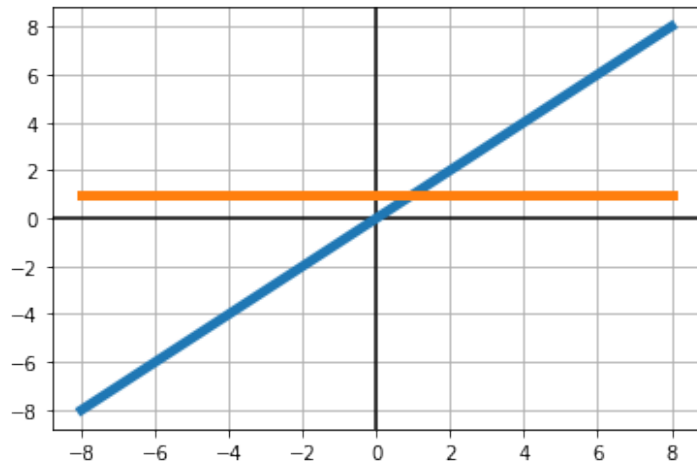
Neuron is a linear function of its input followed by a (typically) non-linear activation function to produce output. $a = \sigma(\mathbf{w}^T \mathbf{x} + b)$

Hyperparameters : activation function (σ)

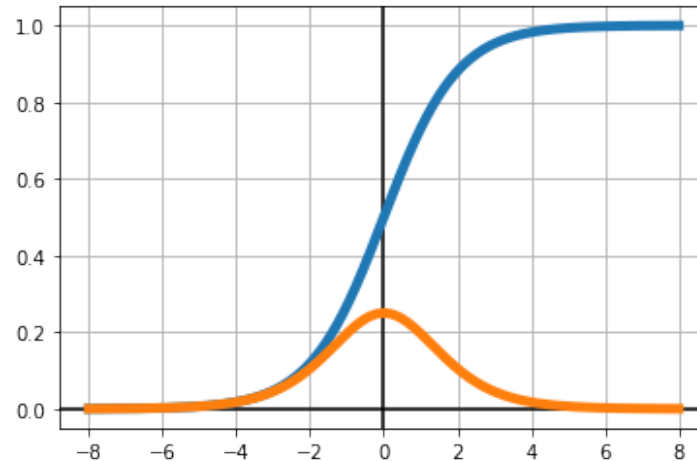
Learnable Parameters: $\mathbf{w} \in \mathbb{R}^d$ and $\mathbf{b} \in \mathbb{R}$



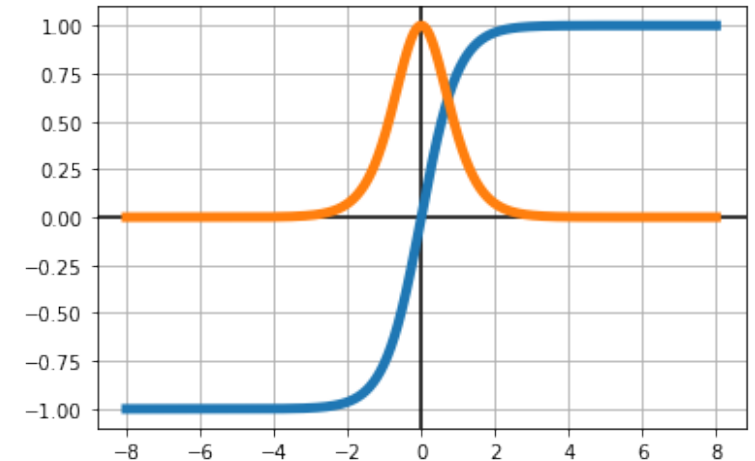
None $\sigma(z) = z$



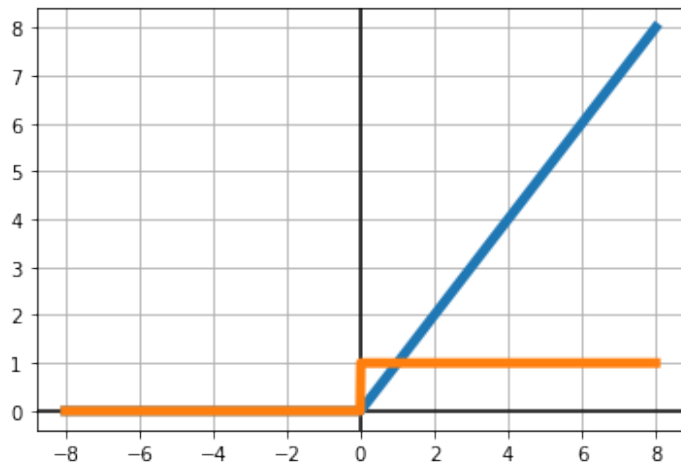
Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$



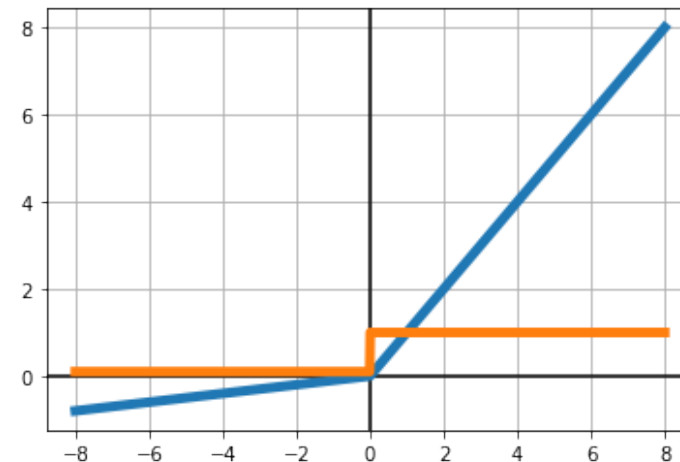
tanh $\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$



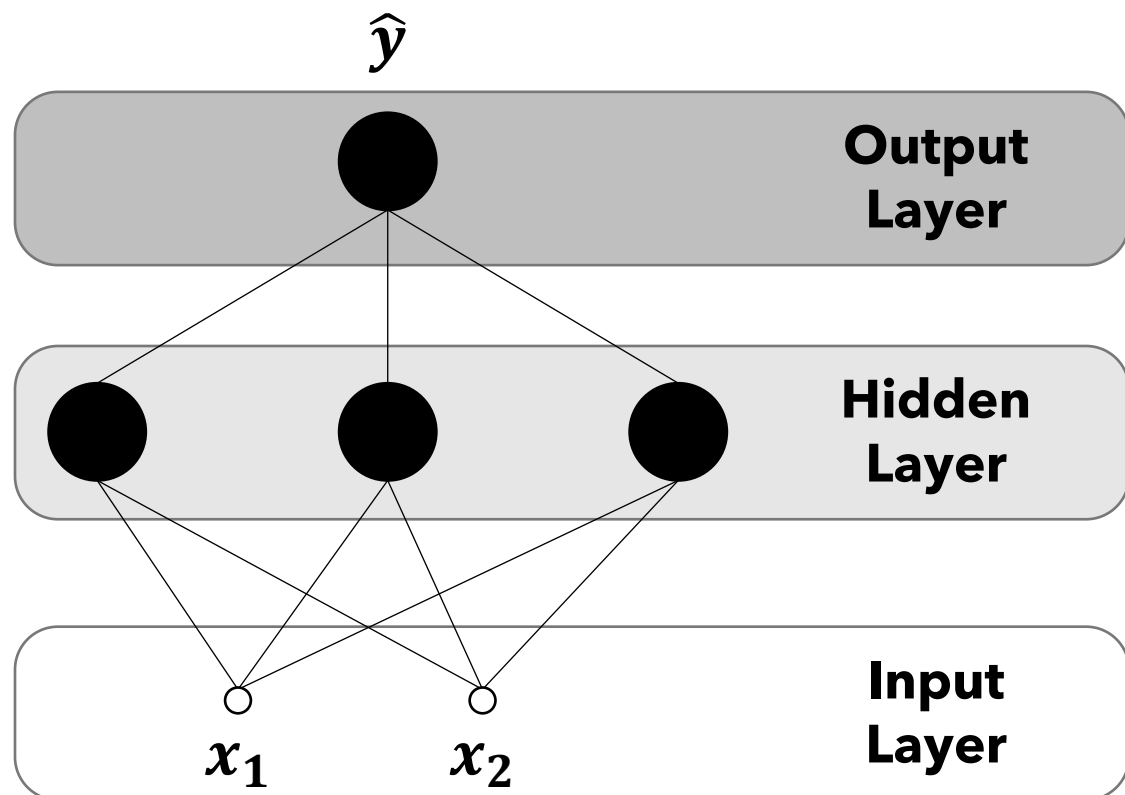
ReLU $\sigma(z) = \max(0, z)$



Leaky ReLU $\sigma(z) = \begin{cases} \alpha z & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$



A Neural Network is a set of connected neurons. A very typical arrangement is a feed-forward multilayer neural network like the one shown below.



Each layer receives its input from the previous layer and forwards its output to the next – thus the **feed-forward** description.

The layers of neurons between the input and output are referred to as **hidden layers**.

- This network for instance is a **2-layer neural** network (1 hidden and 1 output)
- Number of neurons in a layer is referred to as its width.

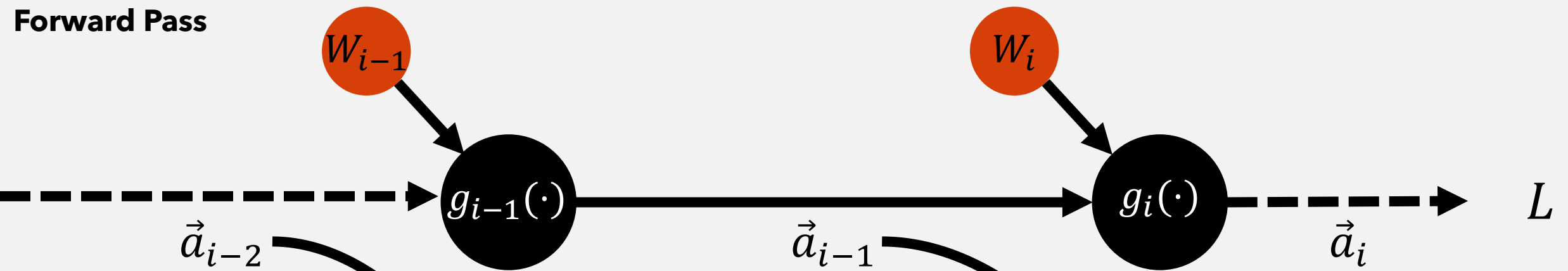
Activation functions in different layers can be heterogenous.

- Output layer's activation is task dependent
 - Linear for regression
 - Sigmoid or softmax for classification

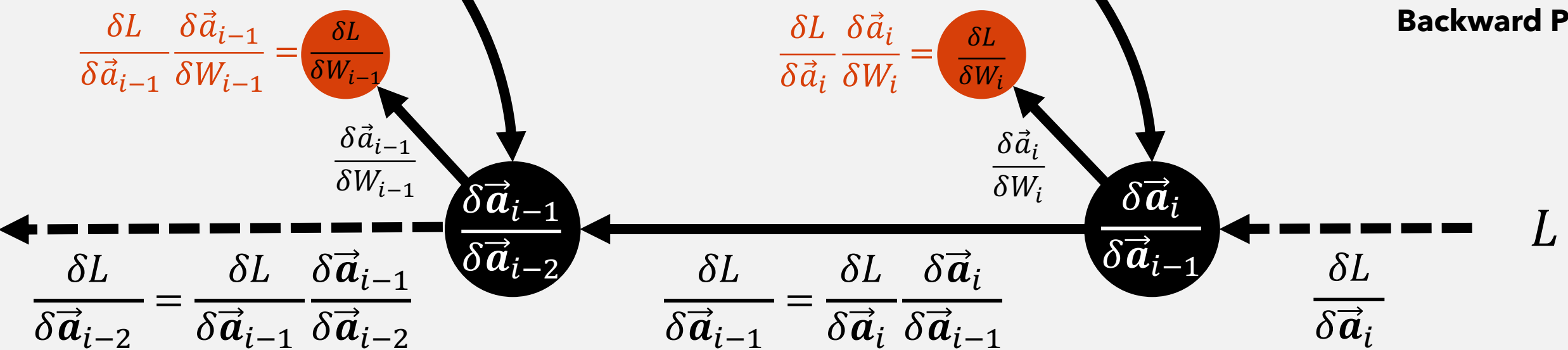


Backpropagation (aka Reverse Mode Auto-differentiation)

Forward Pass



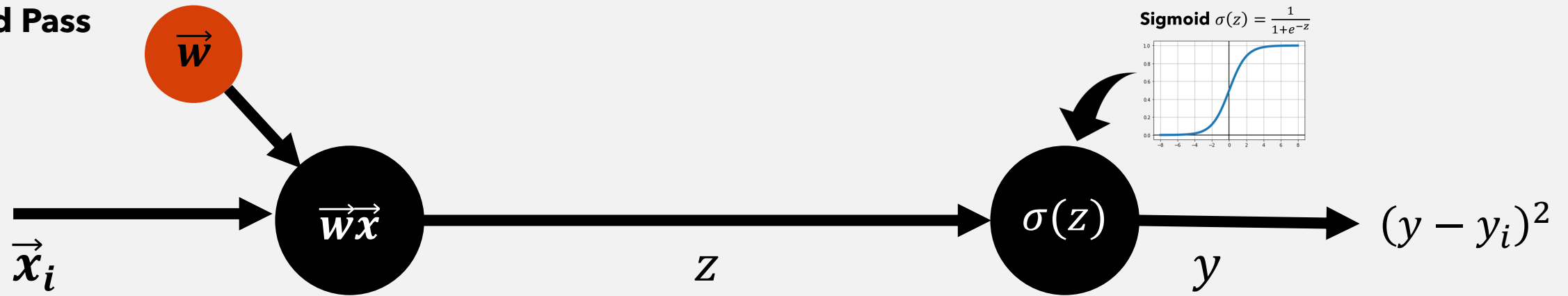
Backward Pass





Backpropagation (aka Reverse Mode Auto-differentiation)

Forward Pass

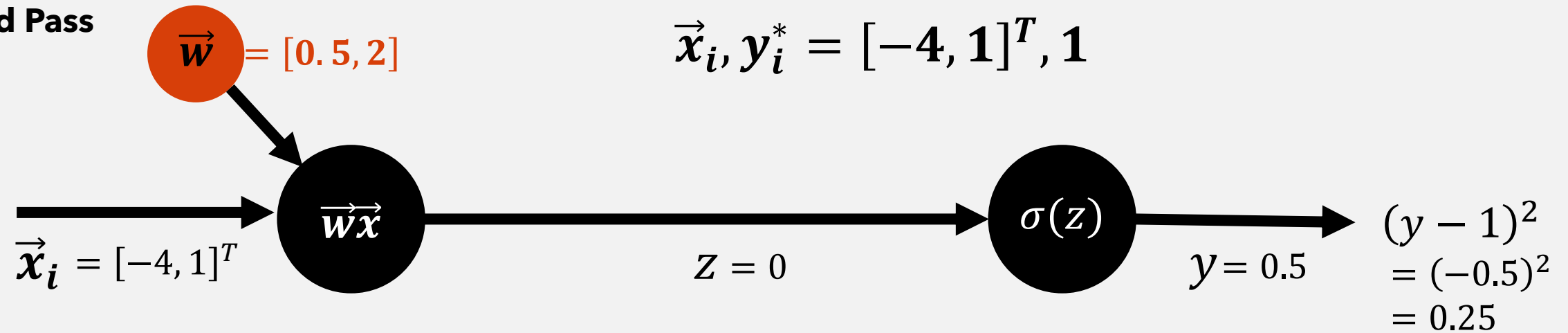


Backward Pass



Backpropagation (aka Reverse Mode Auto-differentiation)

Forward Pass

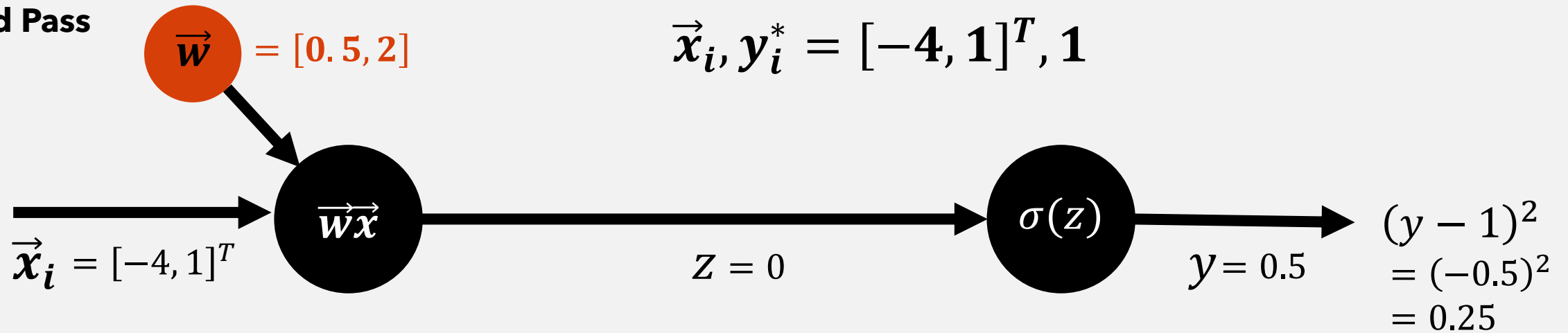


Backward Pass

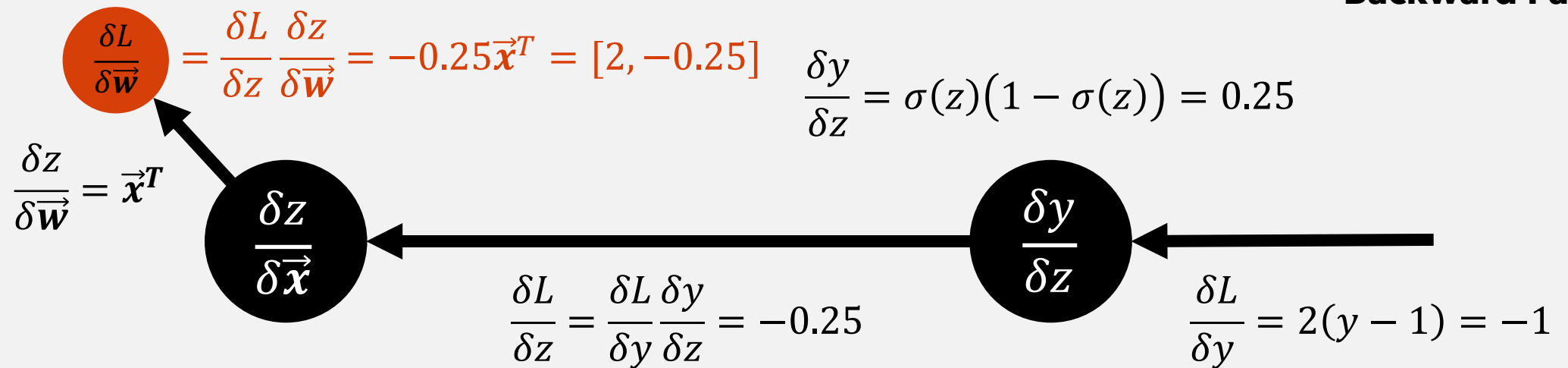


Backpropagation (aka Reverse Mode Auto-differentiation)

Forward Pass



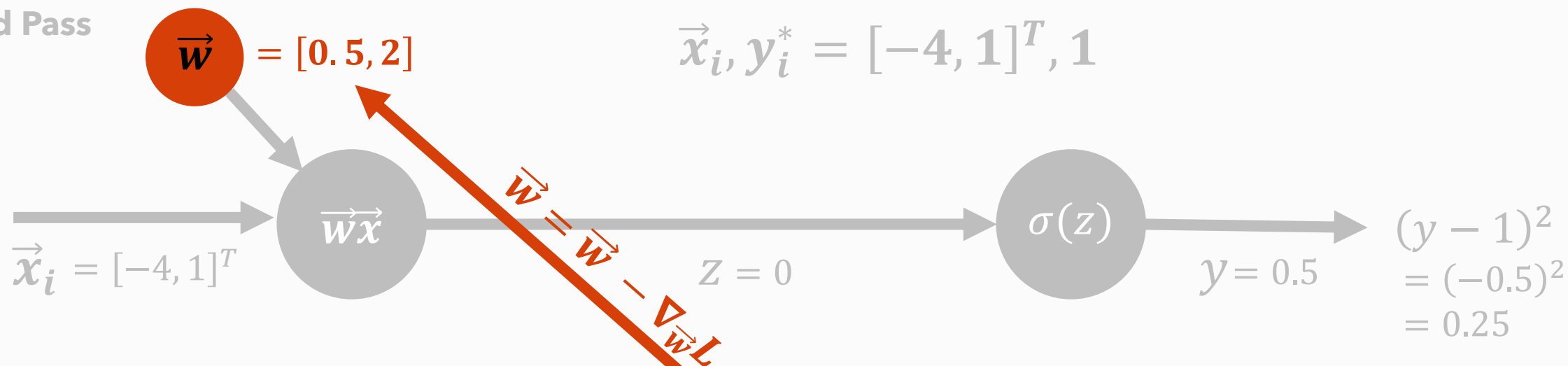
Backward Pass



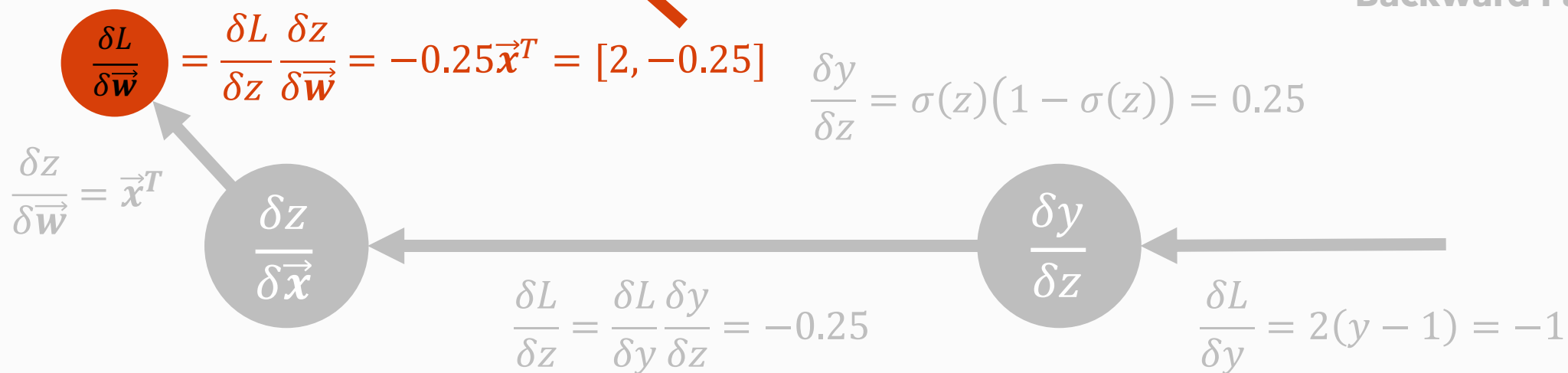


Backpropagation (aka Reverse Mode Auto-differentiation)

Forward Pass



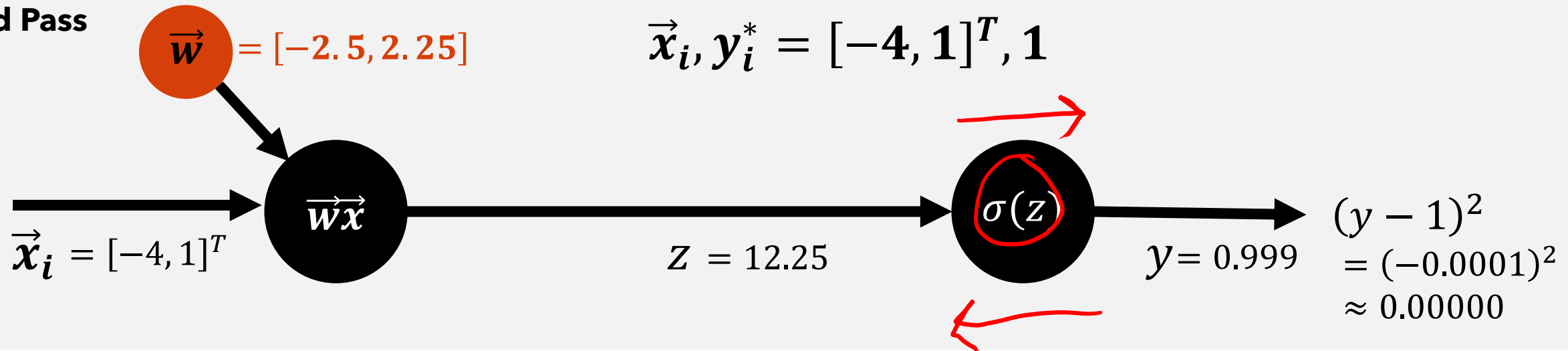
Backward Pass





Backpropagation (aka Reverse Mode Auto-differentiation)

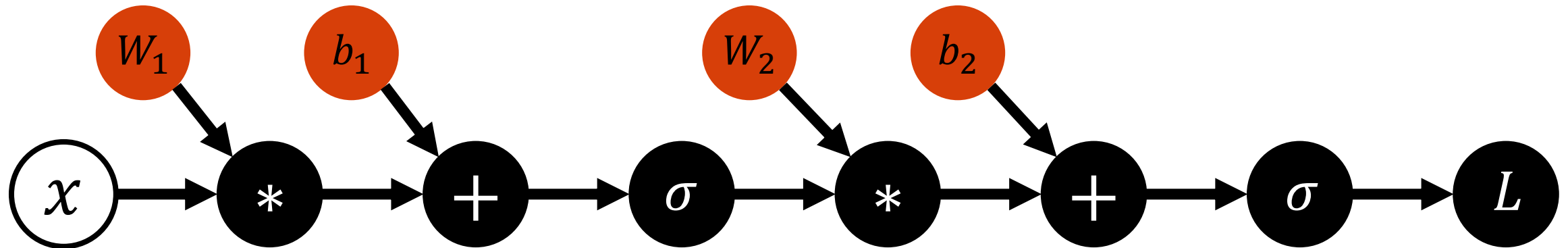
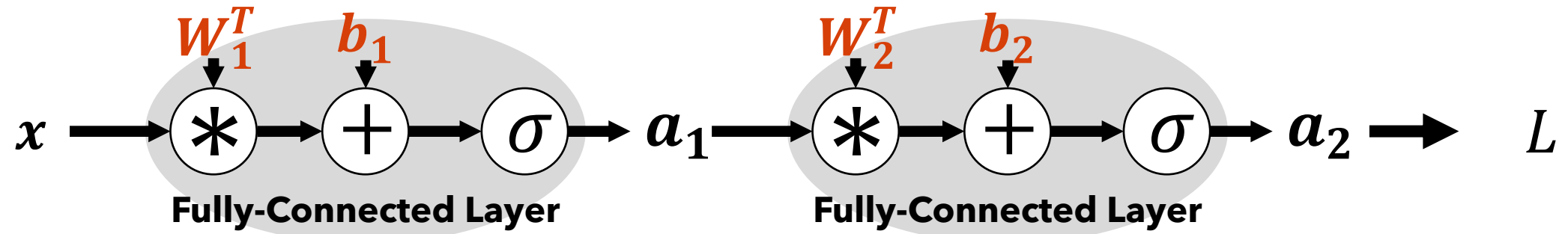
Forward Pass



Backward Pass

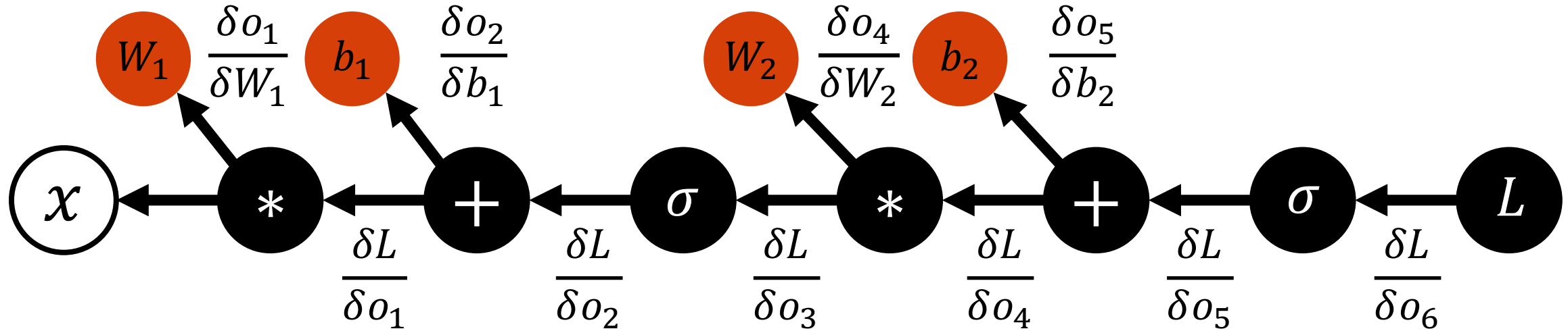
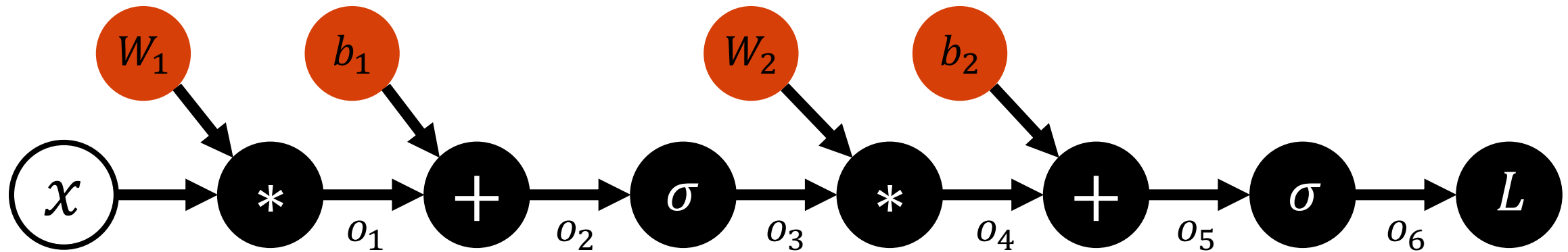


Neural Networks as Computational Graphs





Neural Networks as Computational Graphs



Today's Learning Objectives

Be able to answer:

- What are the intuitions for some advanced structures in neural networks?
- What are decision trees?
- How are they learned?
 - What is entropy, conditional entropy, and information gain?
- How do they deal with continuous features?
- What do to about overfitting?





Intuition 1: Translational Invariance / Locality Assumption



Goal: Classify two 1's followed by two -1's as + and everything else as -

$$\mathbf{x} \in \{-1, 0, 1\}^4$$

$\begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \end{pmatrix}$	✓	$\begin{pmatrix} 0 \\ -1 \\ 0 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 0 \\ -1 \\ -1 \end{pmatrix}$
$\begin{pmatrix} 0 \\ 1 \\ 1 \\ -1 \end{pmatrix}$		$\begin{pmatrix} 0 \\ 1 \\ 0 \\ -1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ -1 \\ 0 \\ 1 \end{pmatrix}$

$$a = \text{sign}(\mathbf{W}^T \mathbf{x} + \mathbf{b})$$

$$\mathbf{W} = \begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \end{pmatrix} \quad \mathbf{b} = \begin{pmatrix} -3.5 \end{pmatrix}$$



Goal: Classify two 1's followed by two -1's as + and everything else as -

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} - 3.5 = 0.5$$

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \begin{bmatrix} 0 \\ 1 \\ 0 \\ -1 \end{bmatrix} - 3.5 = -1.5$$

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \begin{bmatrix} 0 \\ -1 \\ 0 \\ 1 \end{bmatrix} - 3.5 = -6.5$$

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \end{bmatrix} - 3.5 = -2.5$$

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \begin{bmatrix} 0 \\ -1 \\ 0 \\ 1 \end{bmatrix} - 3.5 = -5.5$$

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \begin{bmatrix} 1 \\ 0 \\ -1 \\ -1 \end{bmatrix} - 3.5 = -0.5$$



Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\mathbf{x} \in \{-1, 0, 1\}^8$$

$$a = \max(\text{sign}(\mathbf{W}^T \mathbf{x} + \mathbf{b}))$$

$\begin{pmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 0 \\ -1 \\ -1 \\ 1 \\ 1 \\ -1 \\ -1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 1 \\ 0 \\ -1 \\ 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ -1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ -1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \\ -1 & -1 & 1 & 1 & 0 \\ 0 & -1 & -1 & 1 & 1 \\ 0 & 0 & -1 & -1 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & -1 \end{pmatrix}$	$\begin{pmatrix} -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \end{pmatrix}$
--	---	--	---	---	---	--	--



A Slightly Bigger Toy Problem

Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\mathbf{x} \in \{-1, 0, 1\}^8$$

$$a = \max(\text{sign}(\mathbf{W}^T \mathbf{x} + \mathbf{b}))$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \\ -1 & -1 & 1 & 1 & 0 \\ 0 & -1 & -1 & 1 & 1 \\ 0 & 0 & -1 & -1 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & -1 \end{pmatrix}^T \begin{pmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{pmatrix} + \begin{pmatrix} -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \end{pmatrix} = \begin{pmatrix} 1 - 3.5 \\ 4 - 3.5 \\ 0 - 3.5 \\ -4 - 3.5 \\ -1 - 3.5 \end{pmatrix} \xrightarrow{\max(\text{sign}(\cdot))} 1$$



A Slightly Bigger Toy Problem

Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\mathbf{x} \in \{-1, 0, 1\}^8$$

$$a = \max(\text{sign}(\mathbf{W}^T \mathbf{x} + \mathbf{b}))$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \\ -1 & -1 & 1 & 1 & 0 \\ 0 & -1 & -1 & 1 & 1 \\ 0 & 0 & -1 & -1 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & -1 \end{pmatrix}^T \begin{pmatrix} 0 \\ 1 \\ 0 \\ -1 \\ 1 \\ 1 \\ 1 \\ 0 \end{pmatrix} + \begin{pmatrix} -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \\ -3.5 \end{pmatrix} = \begin{pmatrix} 2 - 3.5 \\ 1 - 3.5 \\ -3 - 3.5 \\ -2 - 3.5 \\ 1 - 3.5 \end{pmatrix} \xrightarrow{\max(\text{sign}(\cdot))} 0$$



Consider finding the presence of a pattern $[1, 1, -1, -1]^T$ in a d dimensional vector $\mathbf{x} \in \{-1, 0, 1\}^d$. How many columns would our weight matrix need to have if we follow the same schema as before?

$$\mathbf{x} \in \{-1, 0, 1\}^4$$

$$\begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \end{pmatrix}$$

$$4 \times 1$$

$$\mathbf{x} \in \{-1, 0, 1\}^8$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \\ -1 & -1 & 1 & 1 & 0 \\ 0 & -1 & -1 & 1 & 1 \\ 0 & 0 & -1 & -1 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & -1 \end{pmatrix}$$

$$8 \times 5$$

$$\mathbf{x} \in \{-1, 0, 1\}^d$$

?

$$d \times ?$$



Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} \\ \\ \\ \\ \\ \\ \\ \end{bmatrix}$$



Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \end{bmatrix}^T \quad \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ \\ \\ \\ \\ \\ \\ \end{bmatrix}$$



A Slightly Bigger Toy Problem

Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}^T \quad \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 4 \end{bmatrix}$$



A Slightly Bigger Toy Problem

Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix}^T \quad \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 4 \\ 0 \end{bmatrix}$$



A Slightly Bigger Toy Problem

Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix}^T \quad \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 4 \\ 0 \\ -4 \end{bmatrix}$$



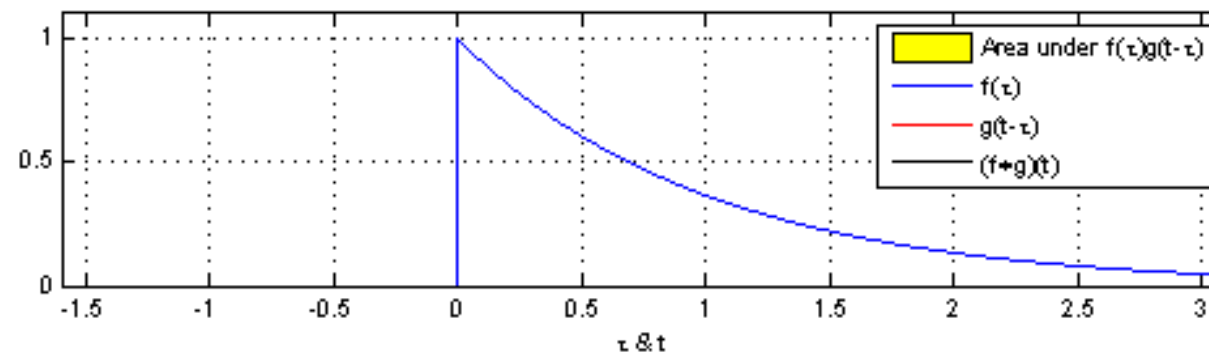
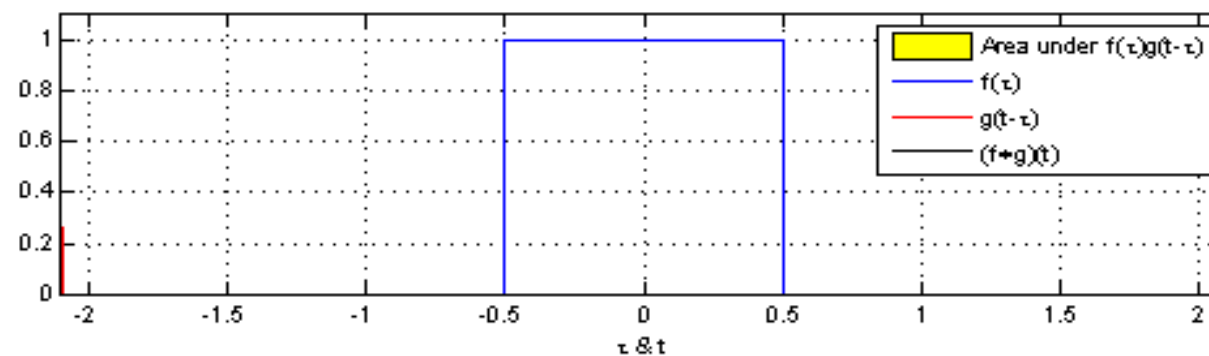
A Slightly Bigger Toy Problem

Goal: Classify any signal with two 1's followed by two -1's as positive.

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} -1 \\ 1 \\ 1 \\ 0 \end{bmatrix}^T \quad \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 4 \\ 0 \\ -4 \\ -1 \end{bmatrix}$$



$$(f \circledast g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$





$$(f \circledast g)(t) = \sum_{\tau=0}^N f[t + \tau]g[\tau], \quad g \in \mathbb{R}^N$$

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} * \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \rightarrow \begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ 0 \\ -4 \\ -1 \end{bmatrix}$$



Goal: Classify any signal with two 1's followed by two -1's as positive.

$$a = \max(\text{sign}(\mathbf{W}^T \mathbf{x} + \mathbf{b})) \longrightarrow a = \max(\text{sign}(\mathbf{w} \circledast \mathbf{x} + \mathbf{b}))$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ -1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} -3.5 \\ 0.5 \end{bmatrix}$$

$\mathbf{W}$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

**SAVED PARAMETERS BY MAKING A LOCALITY ASSUMPTION!
ALSO KNOWN AS "TRANSLATIONAL INVARIANCE"**

$$\begin{bmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} -3.5 \\ -3.5 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Goal: Have a feature respond to the word "Alexa"

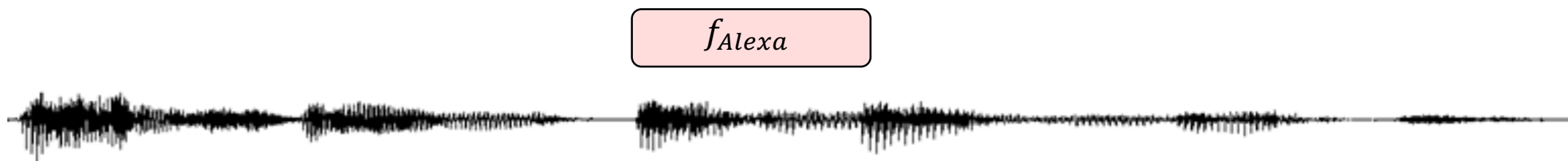
f_{Alexa}



Goal: Have a feature respond to the word "Alexa"



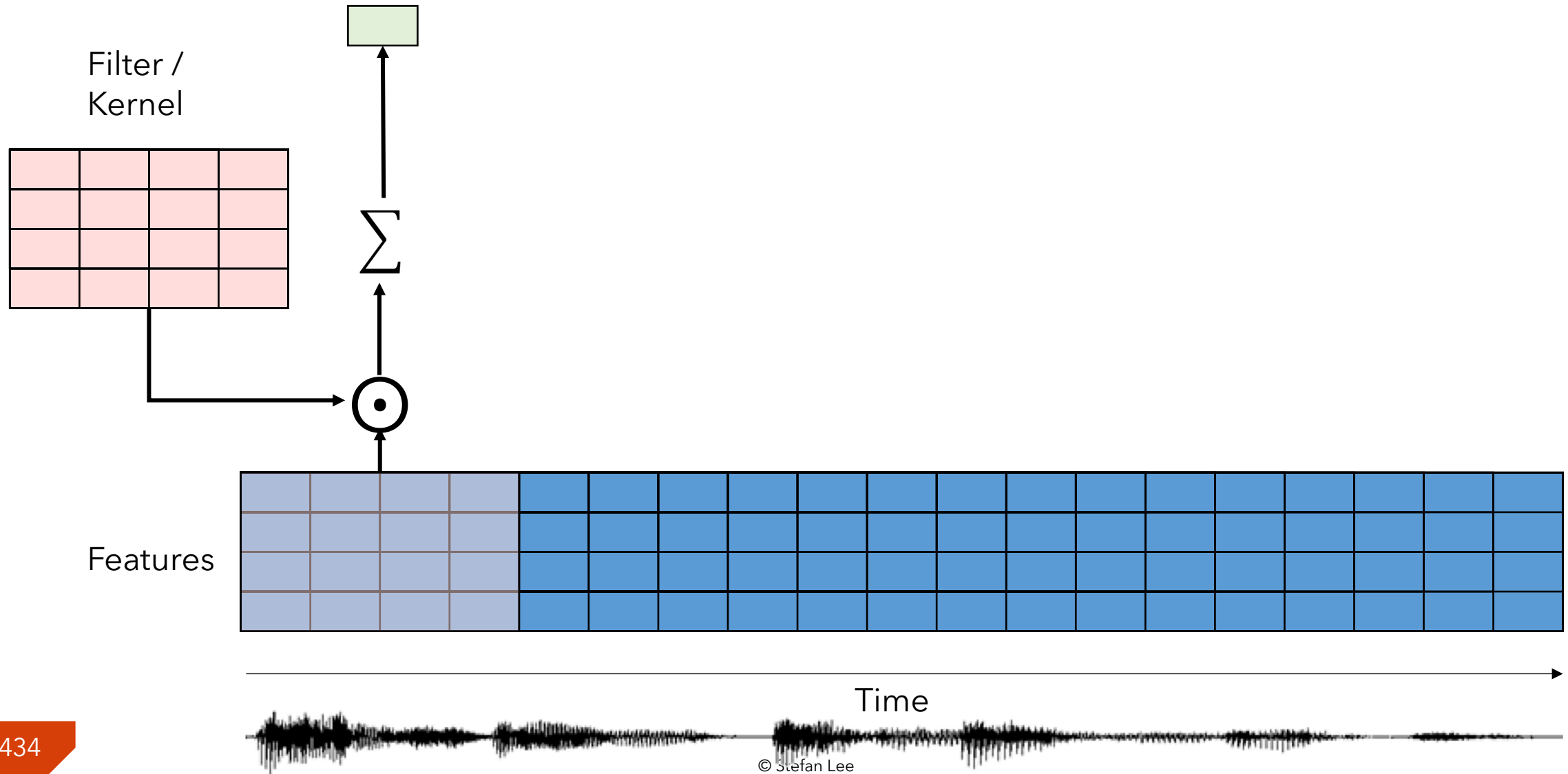
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

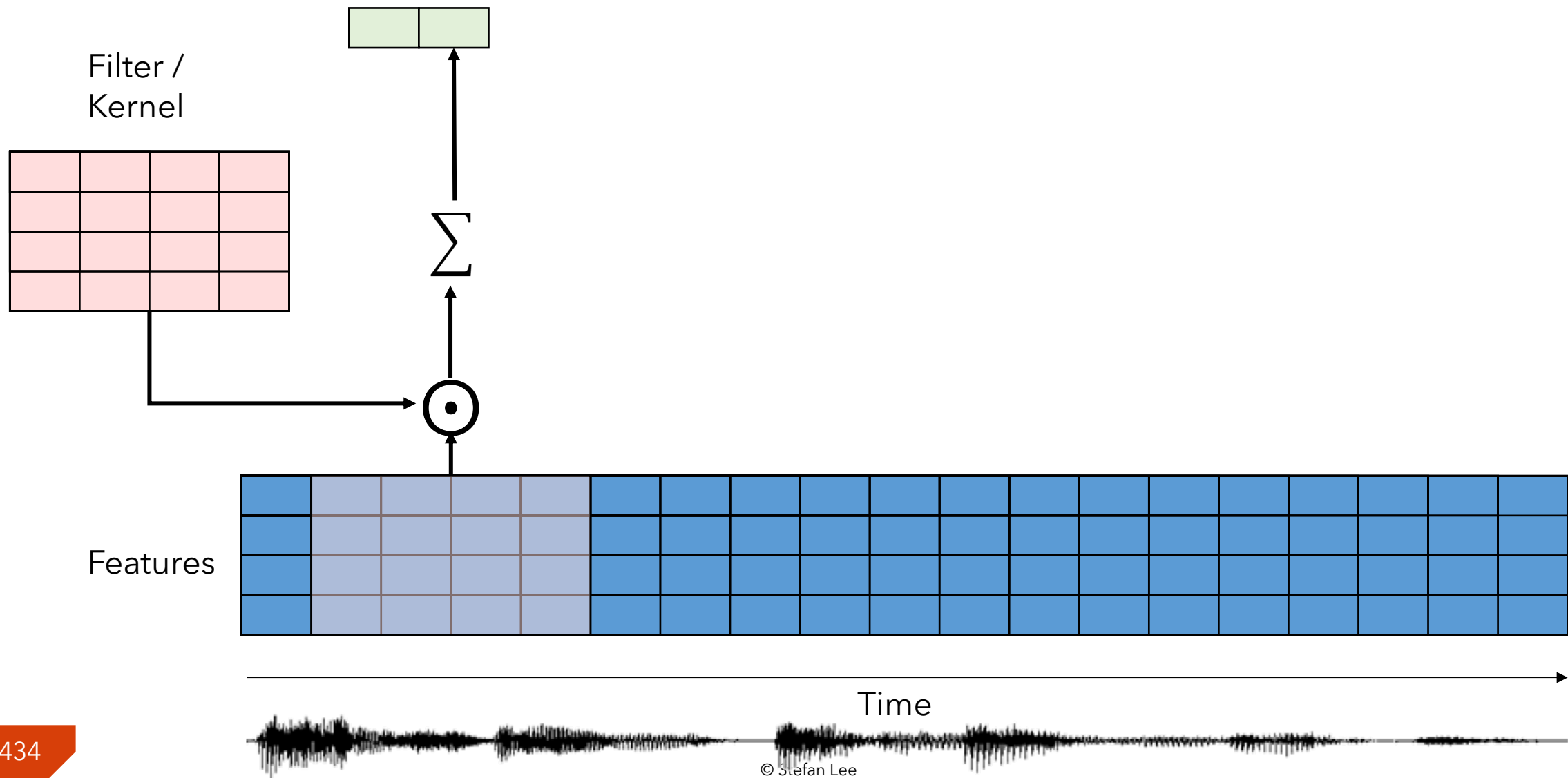
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

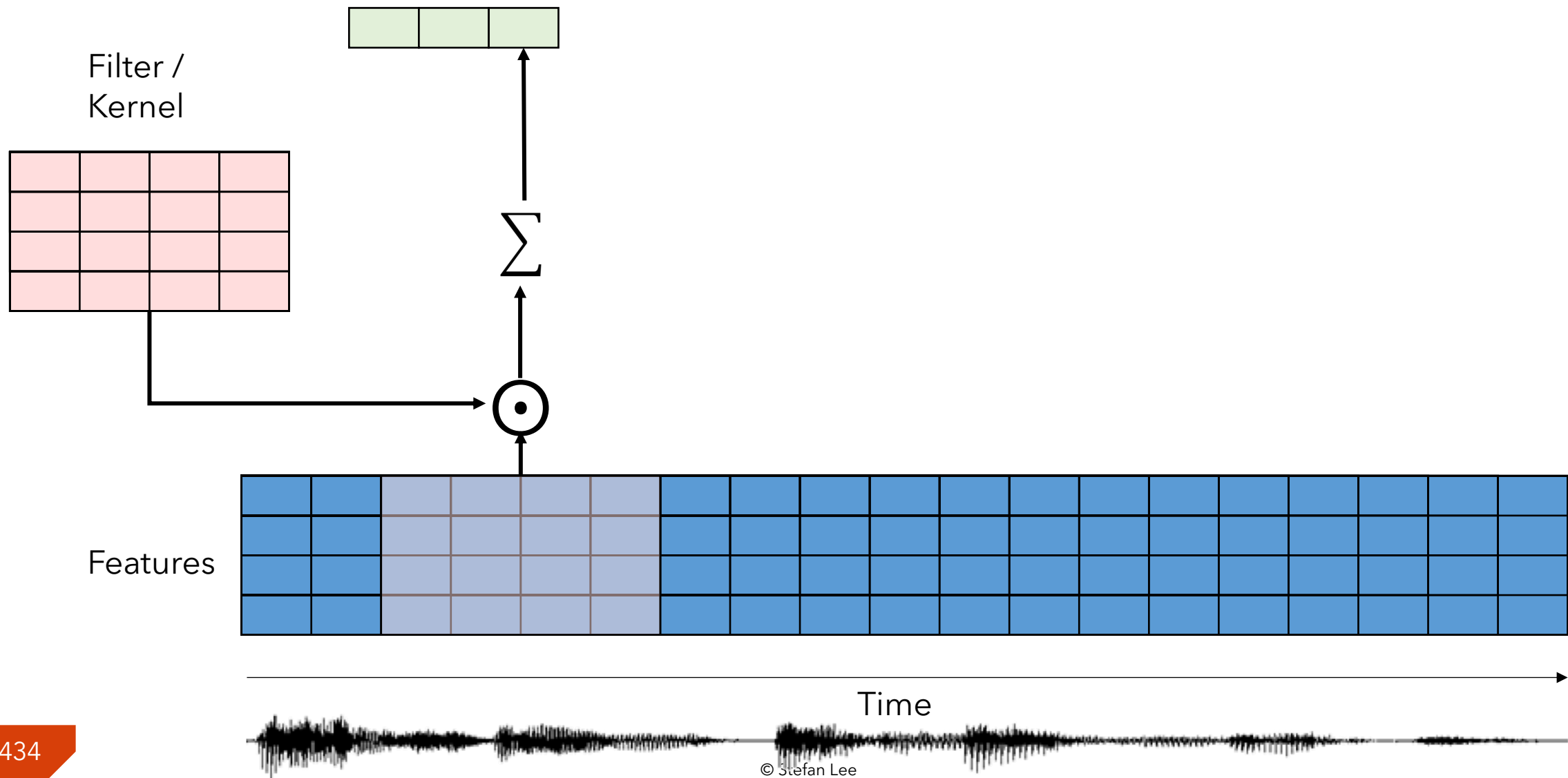
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

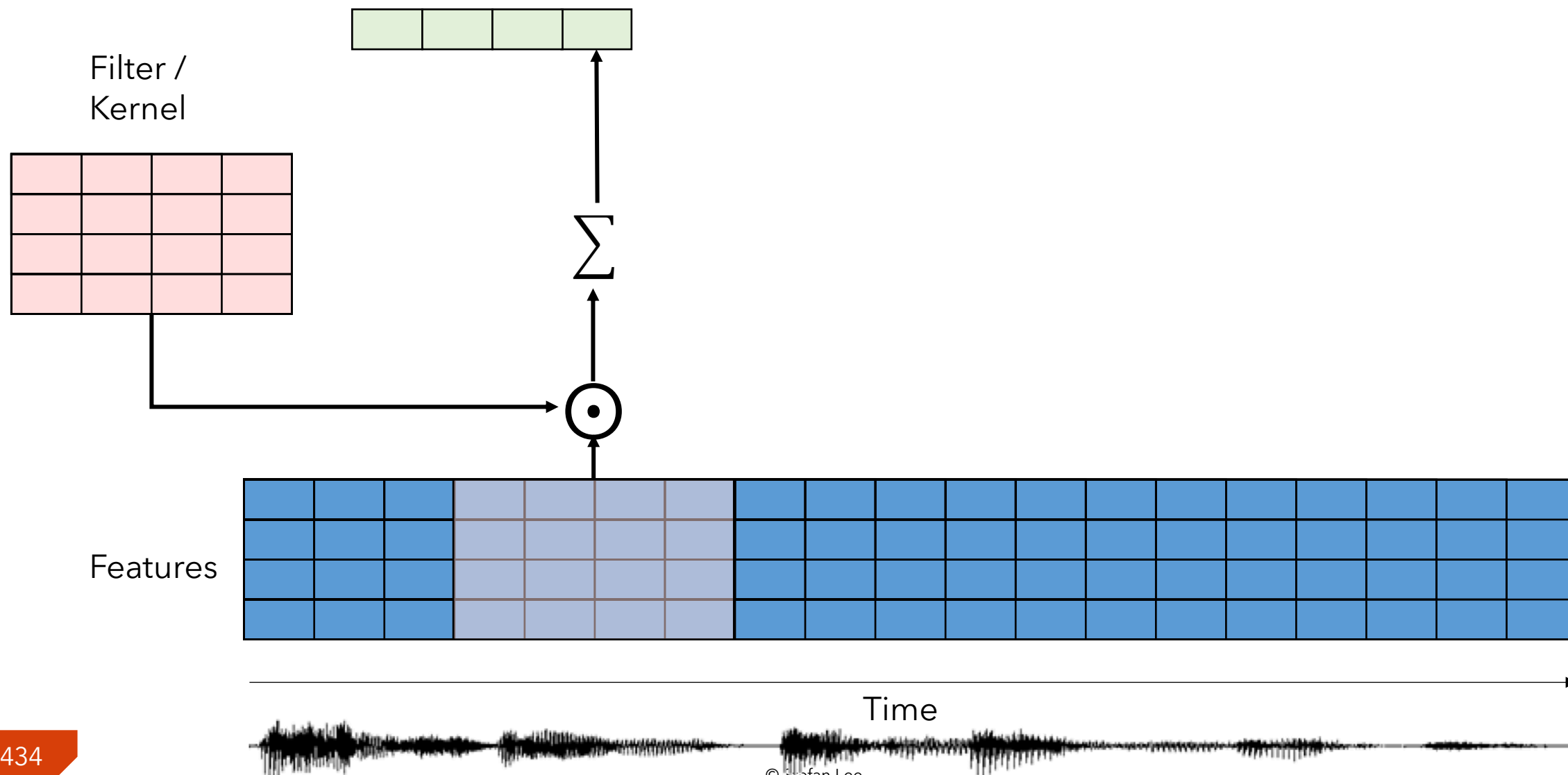
Goal: Have a feature respond to the word "Alexa"





Example where the Locality Bias Fits (1D)

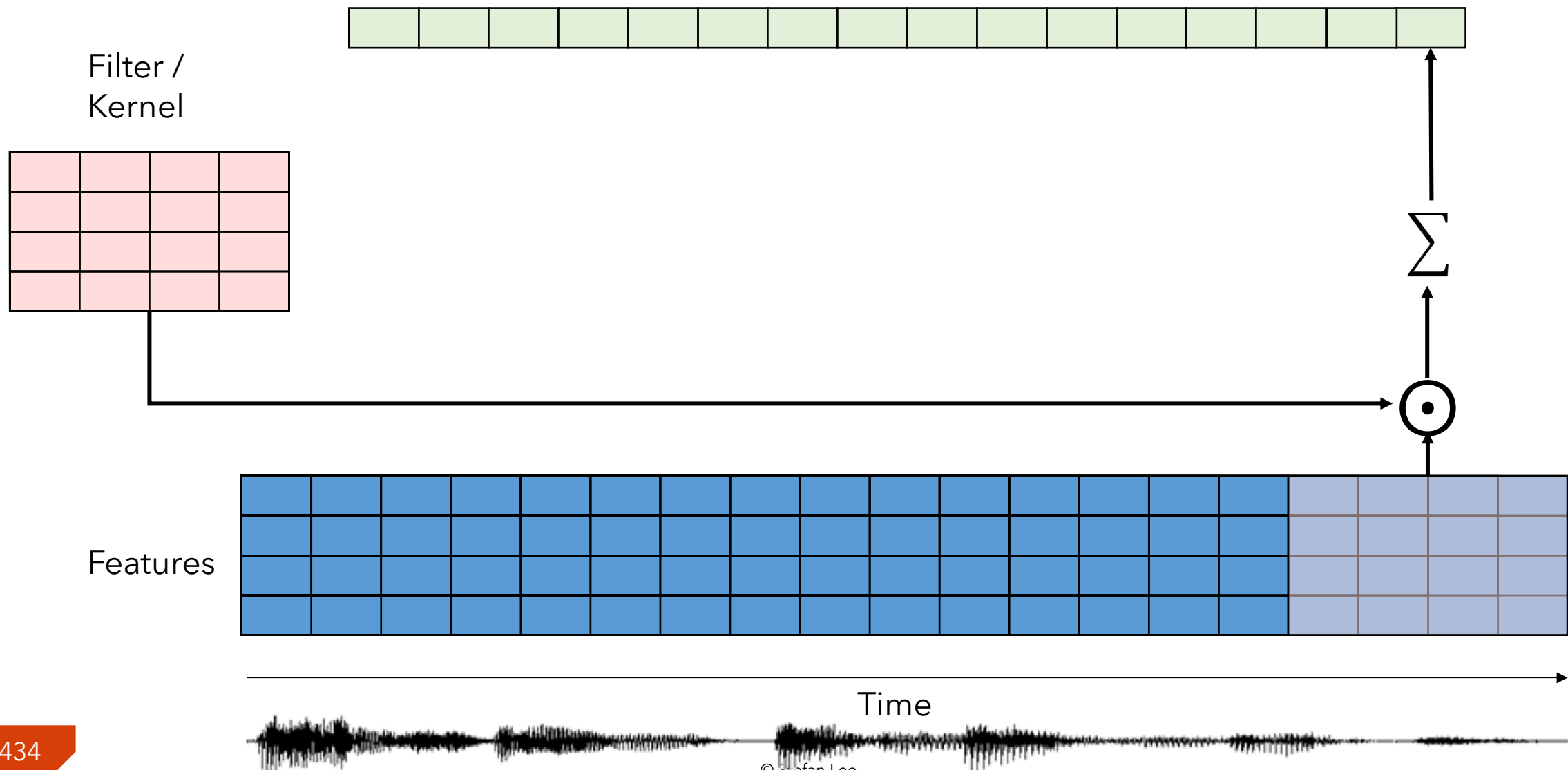
Goal: Have a feature respond to the word "Alexa"



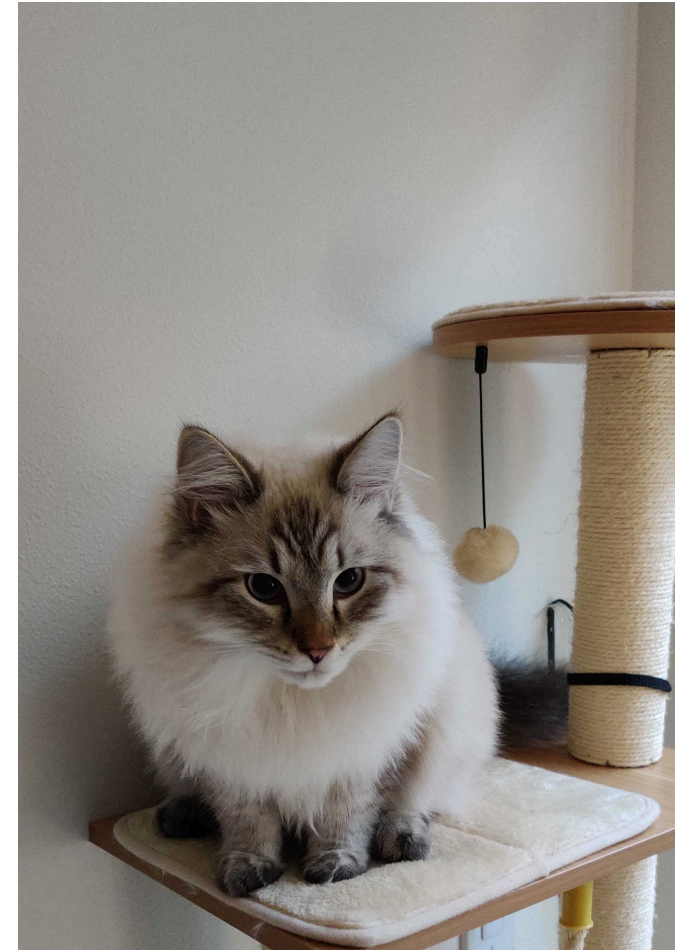
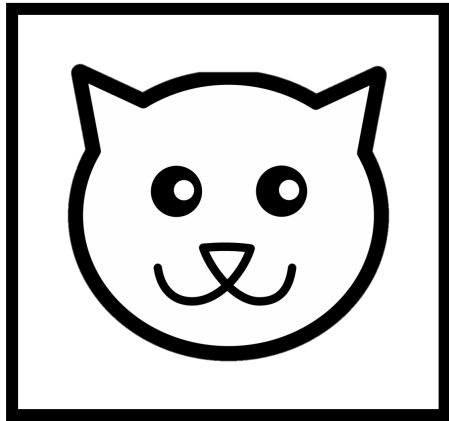


Example where the Locality Bias Fits (1D)

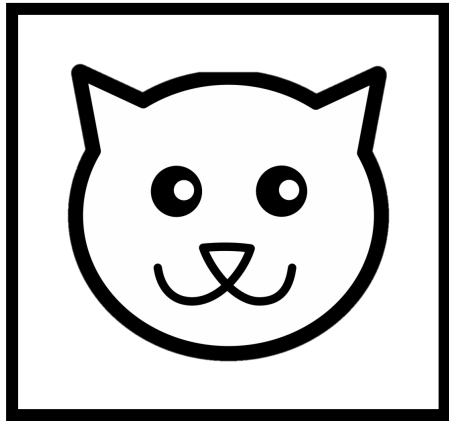
Goal: Have a feature respond to the word "Alexa"

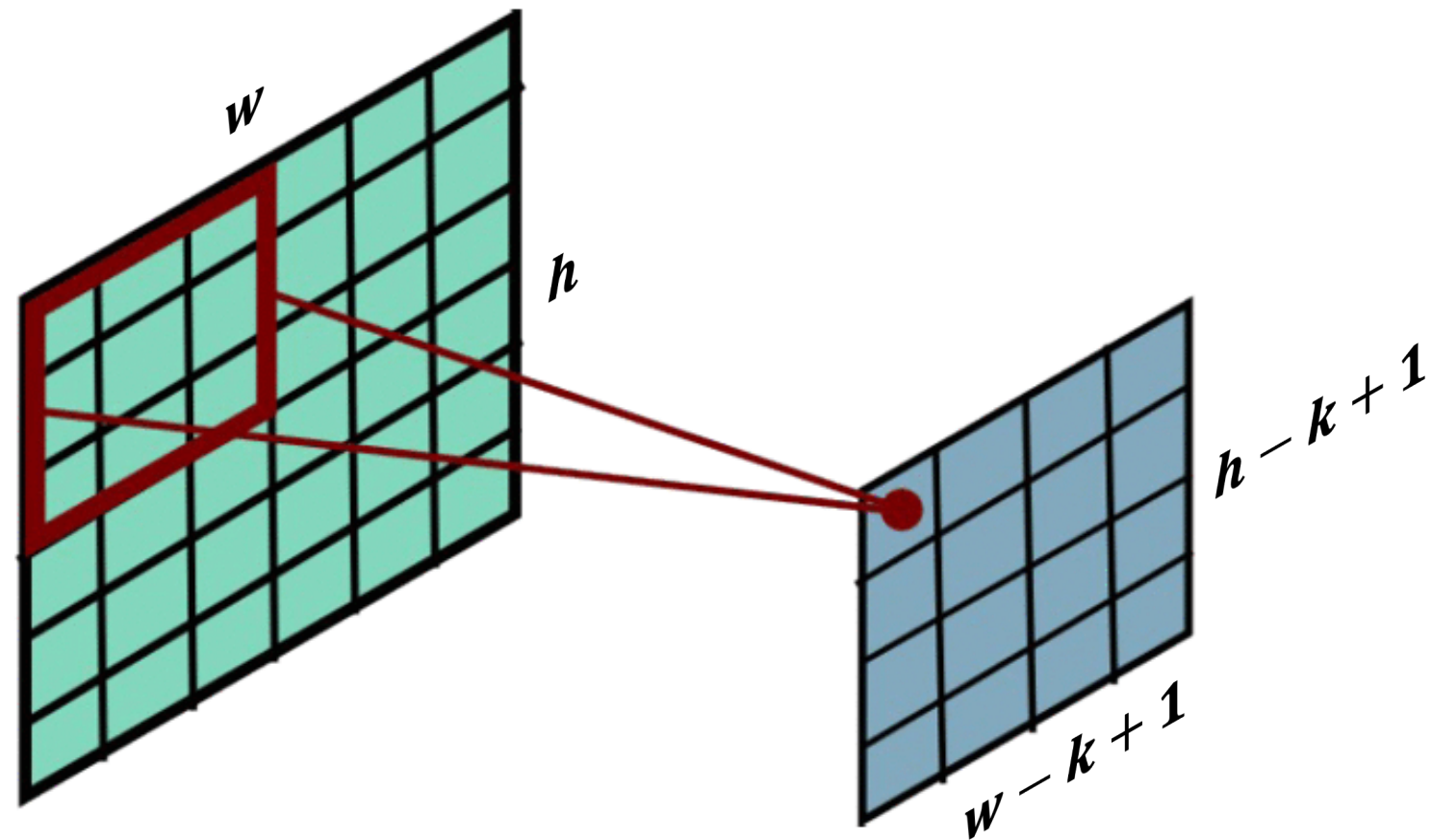


Goal: Have a feature respond to cat faces



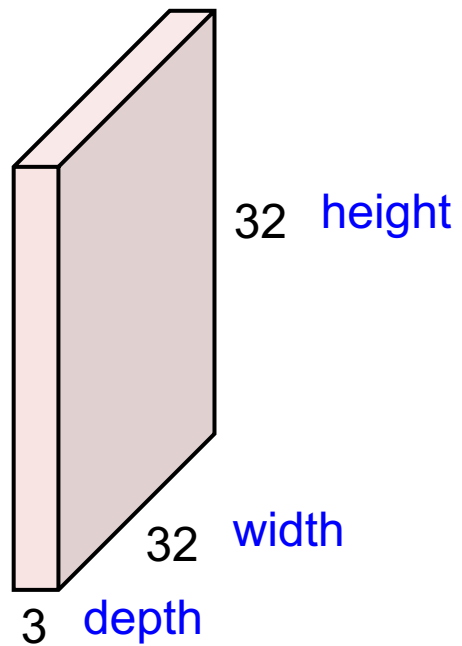
Goal: Have a feature respond to cat faces





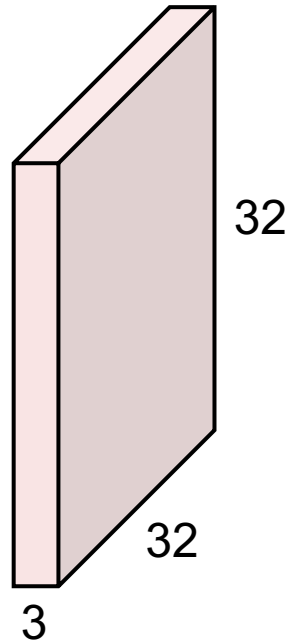


32x32x3 image -> preserve spatial structure





32x32x3 image



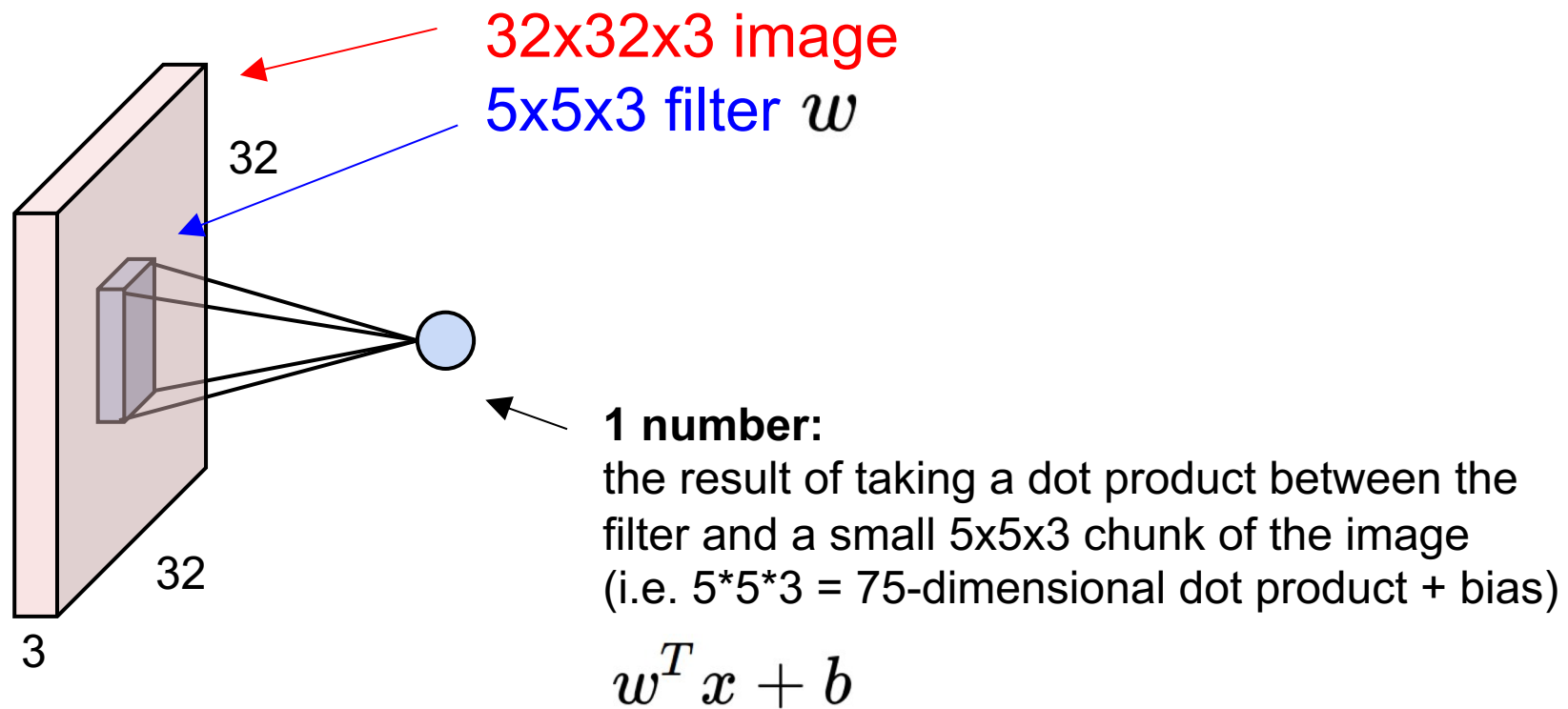
5x5x3 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

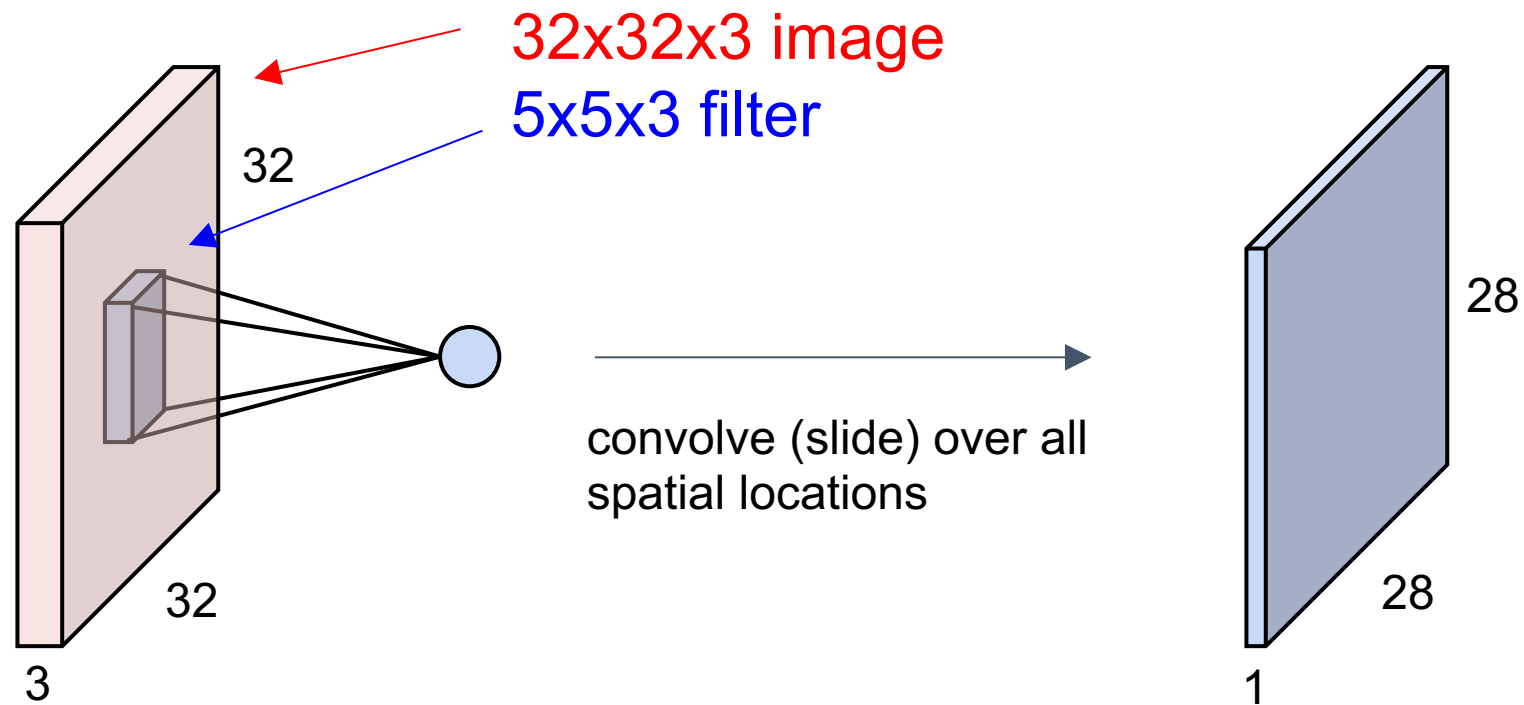


Discrete Convolution in 2D





Discrete Convolution in 2D



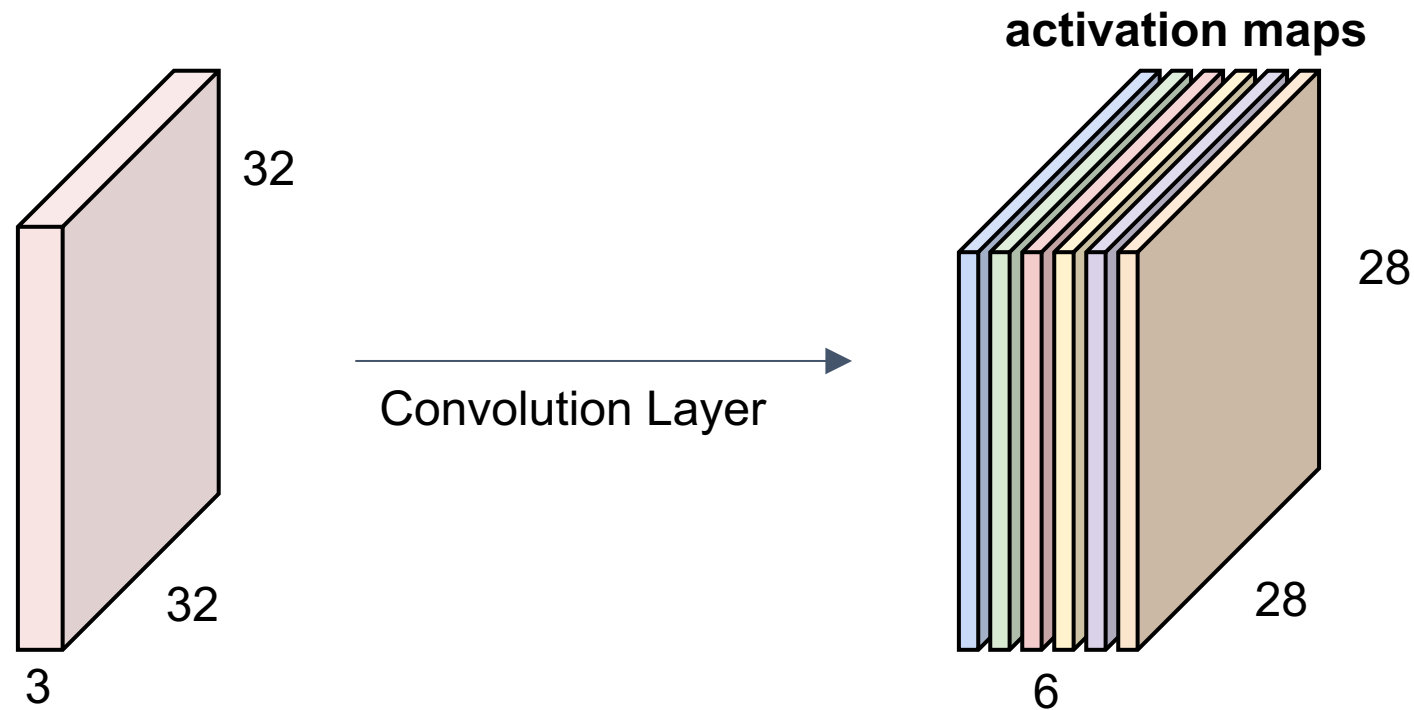


consider a second, **green** filter

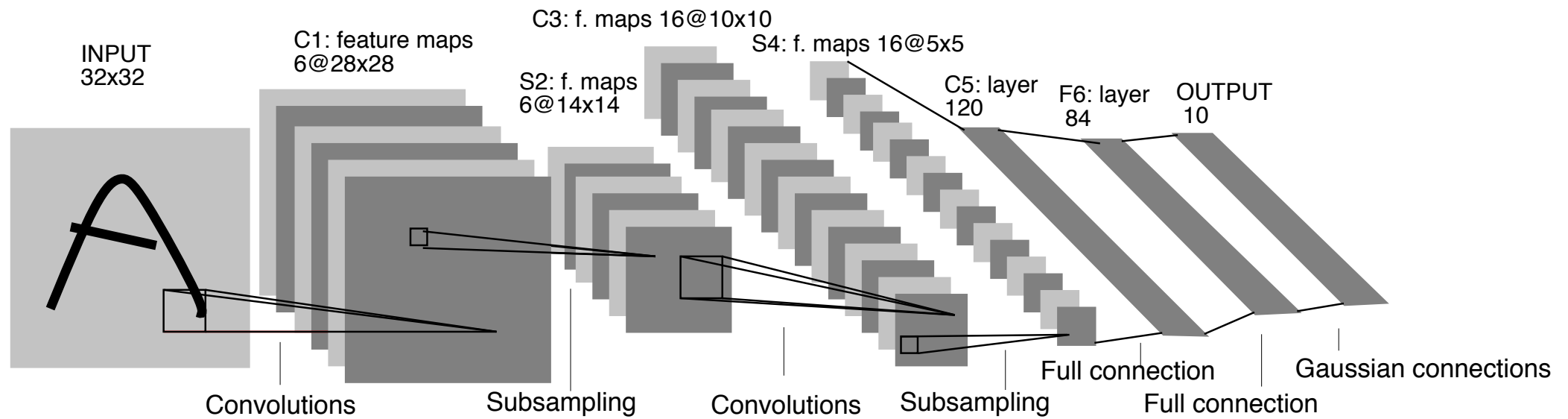




For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

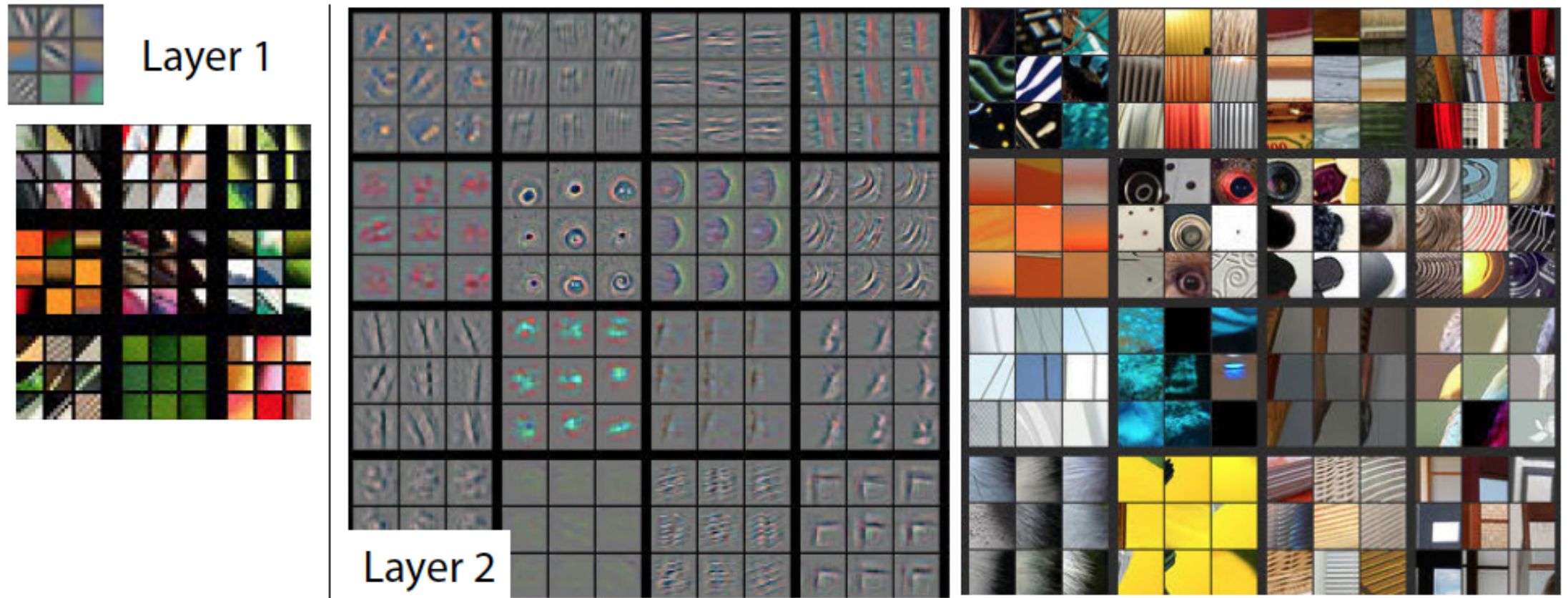


We stack these up to get a “new image” of size 28x28x6!



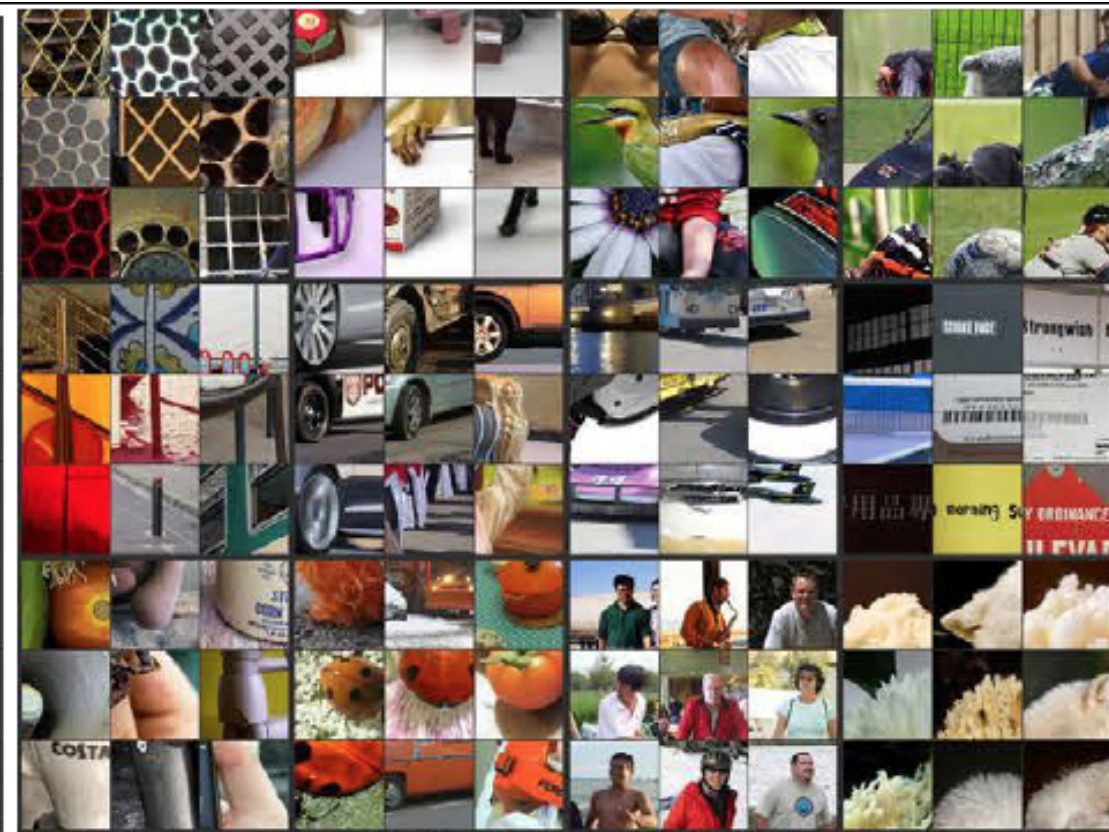
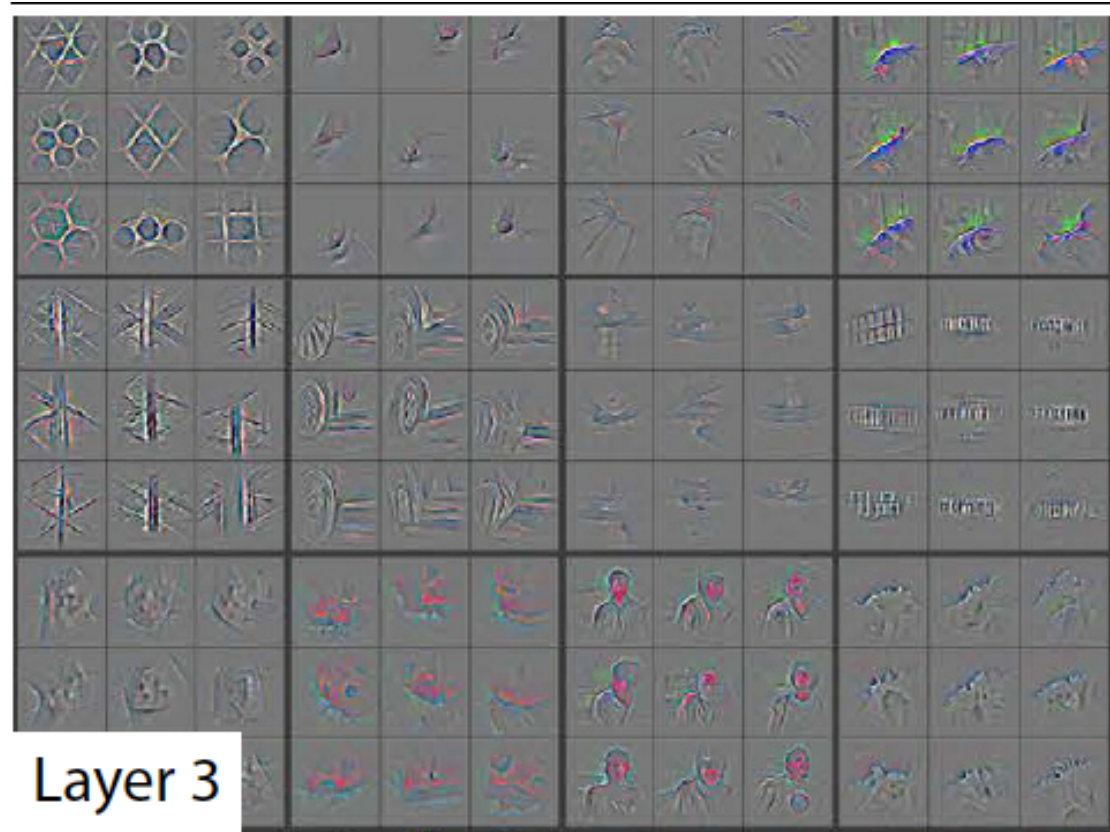


What do convolutional layers learn from images?



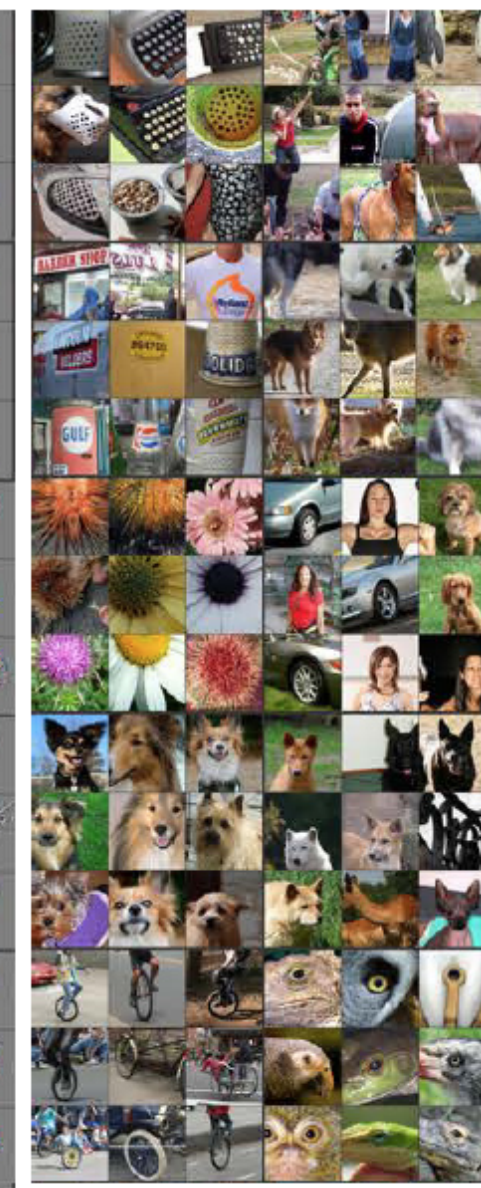
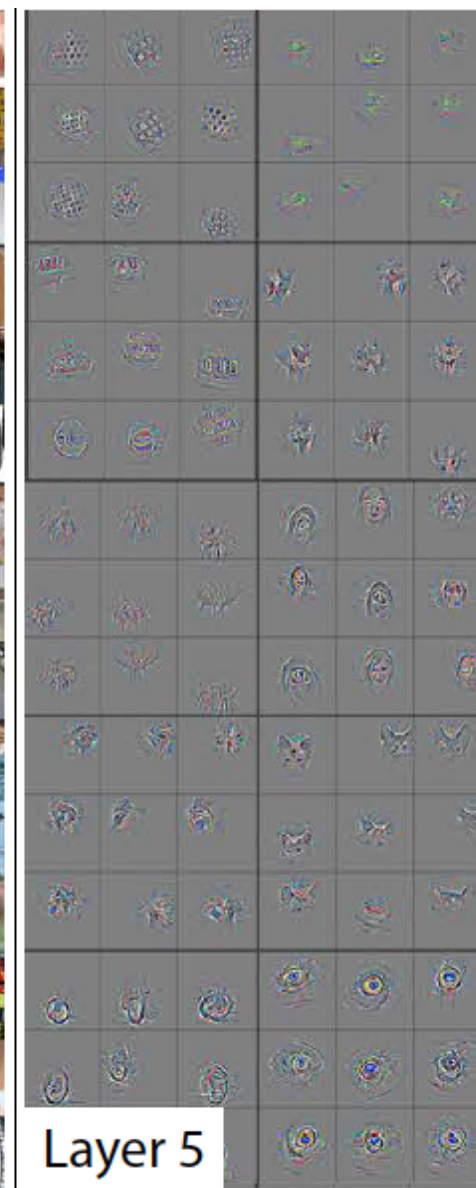


What do convolutional layers learn from images?





What do convolutional layers learn from images?





Intuition 2: Recurrence



Goal: Classify positively if an even number of 1's, negative for odd. $x \in \{0, 1\}^?$

CS 101 Online Parity Algorithm:

$$\begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

```
parity = 0
for bit in x:
    parity = (parity + bit) % 2
return parity
```

Core actions repeated over the sequence:

- Store a memory of the state
- Update that memory based on local information



Goal: Classify positively if an even number of 1's, negative for odd. $x \in \{0, 1\}^?$

CS 102 Online Parity Algorithm:

$$\begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ \vdots \\ 1 \end{pmatrix}$$

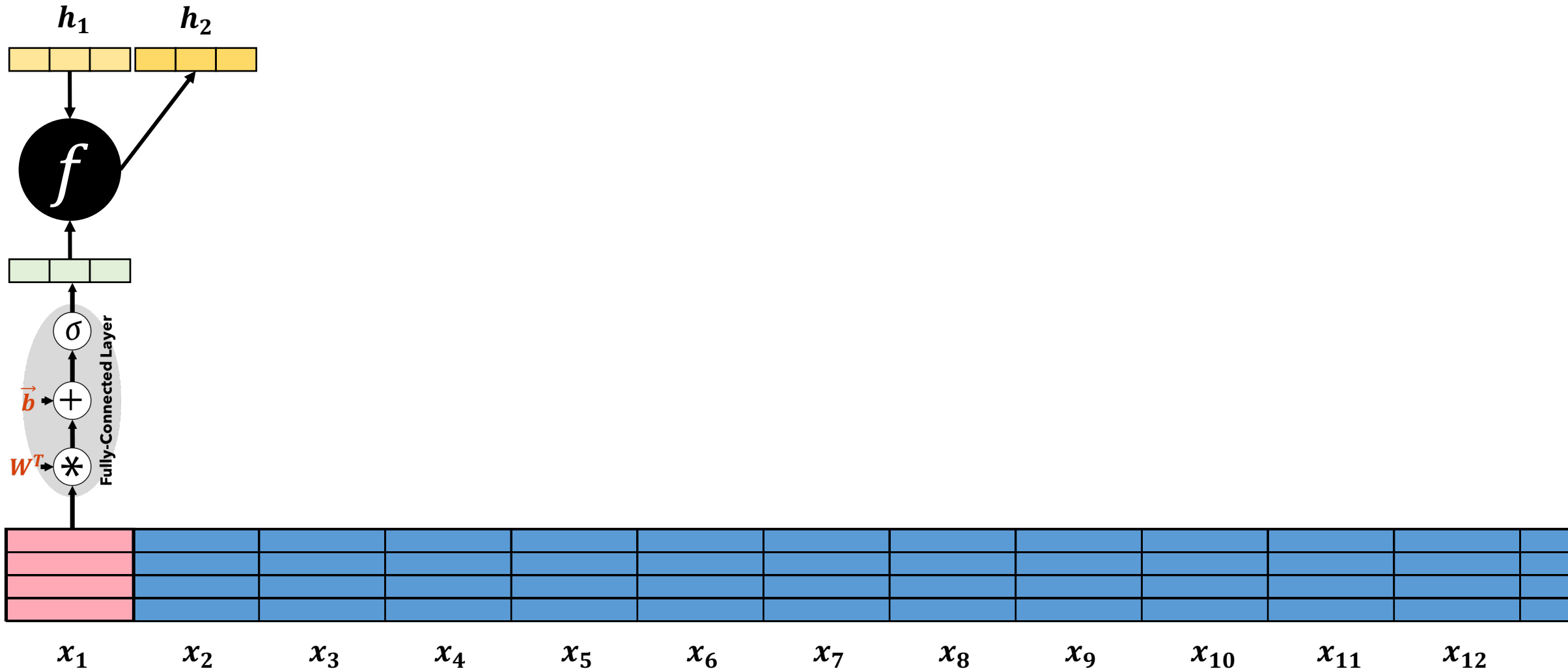
```
def parity(x):  
    if x == None: return 0  
    return (x[-1] + parity(x[:-1]))%2
```

Core actions repeated over the sequence:

- Store a memory of the state
- Update that memory based on local information



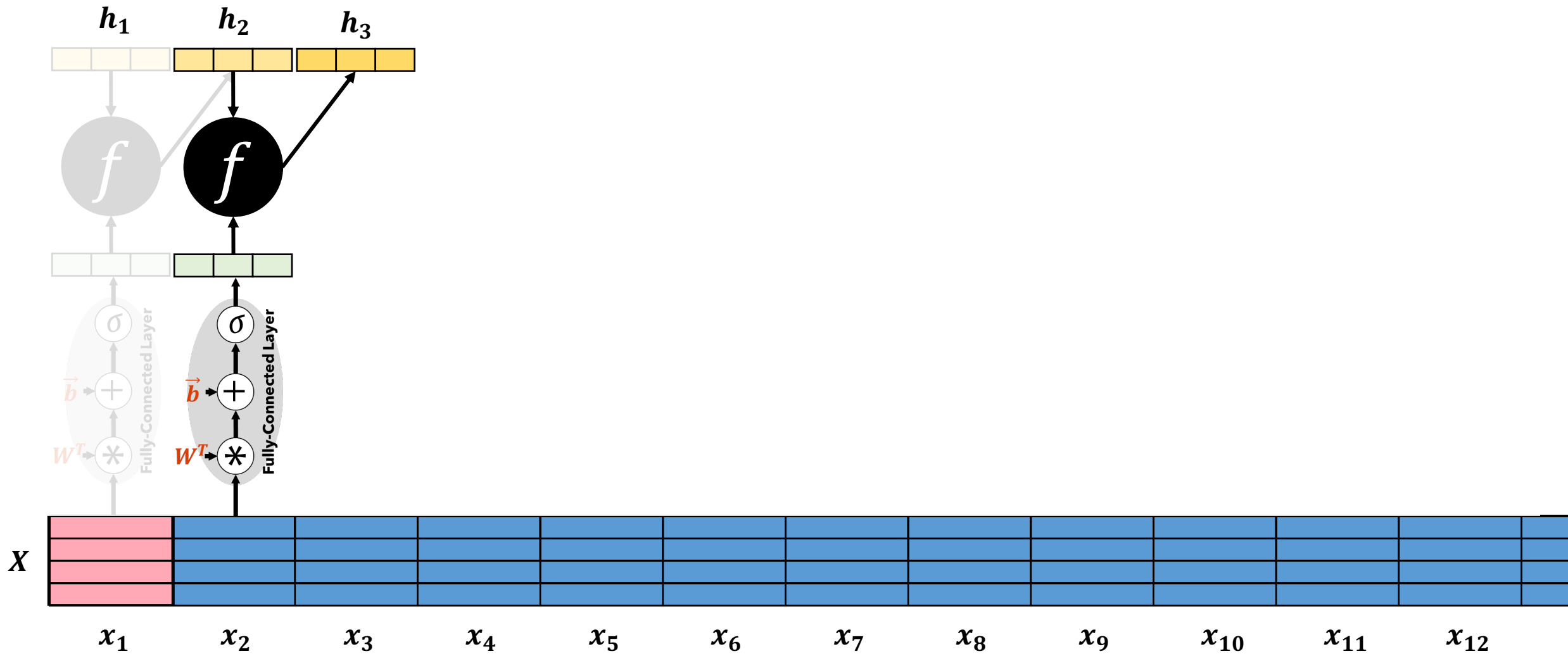
Idea- Keep a memory around and update it with the same function over time.





Variable Input $\rightarrow$ Fixed Output

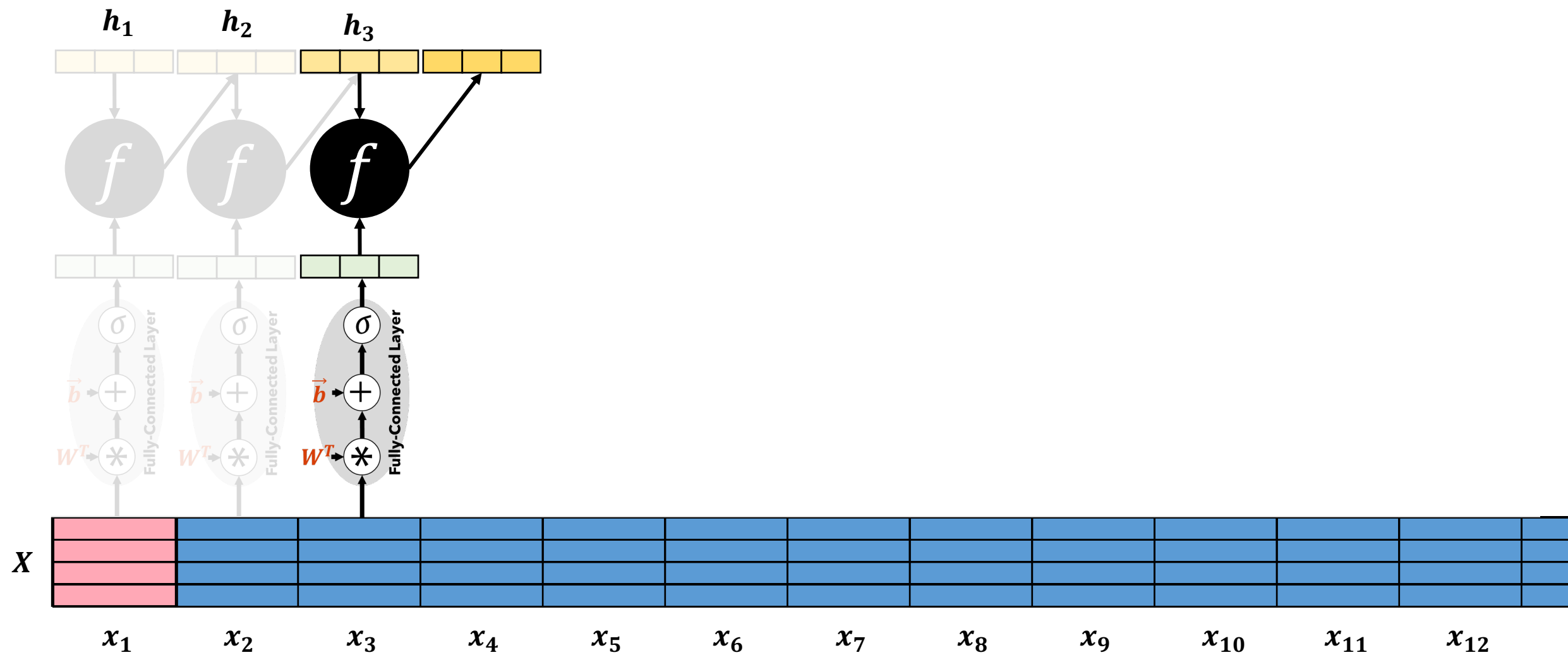
Idea- Keep a memory around and update it with the same function over time.





Variable Input $\rightarrow$ Fixed Output

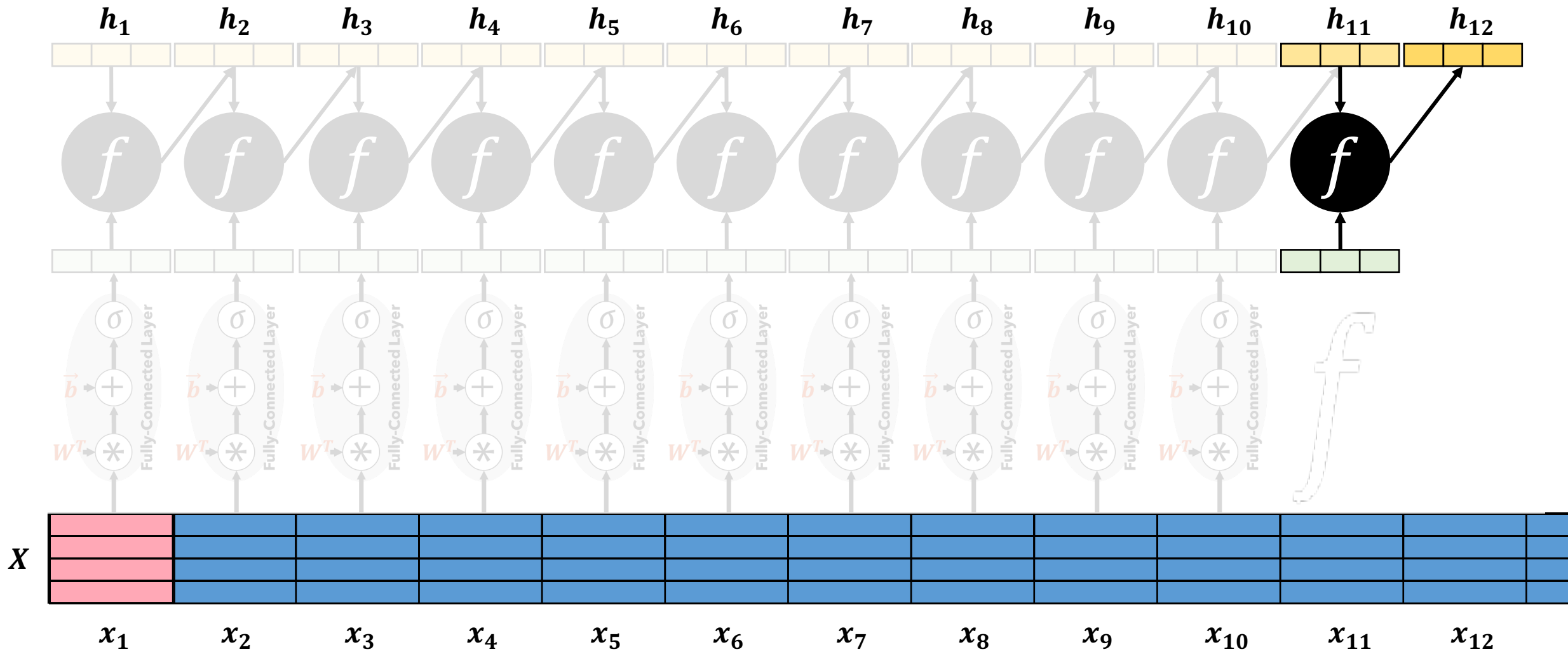
Idea- Keep a memory around and update it with the same function over time.

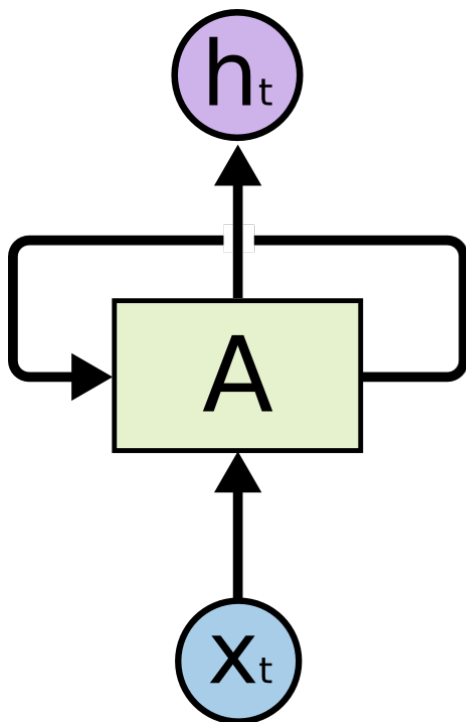


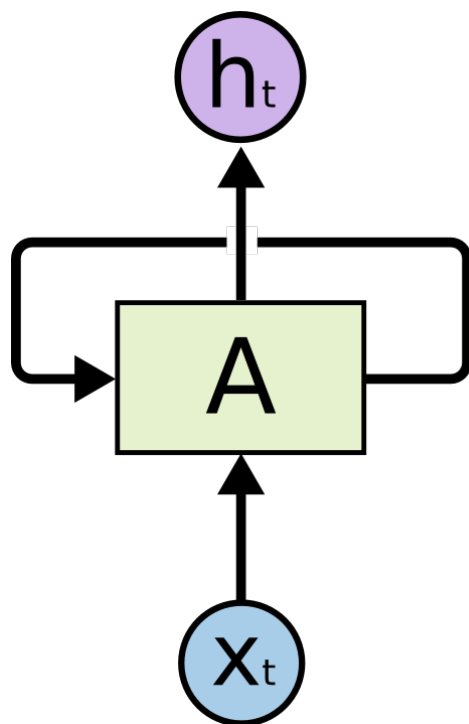


Variable Input $\rightarrow$ Fixed Output

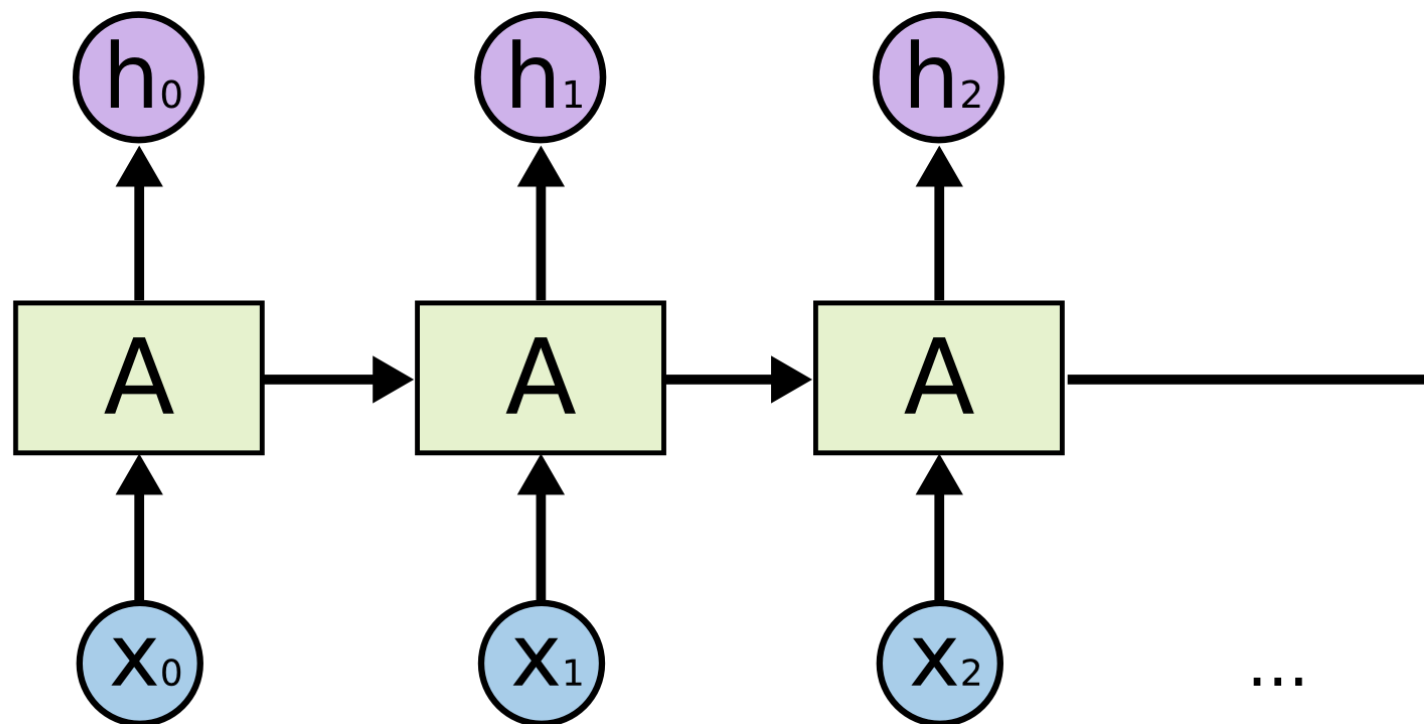
Idea- Keep a memory around and update it with the same function over time.







=

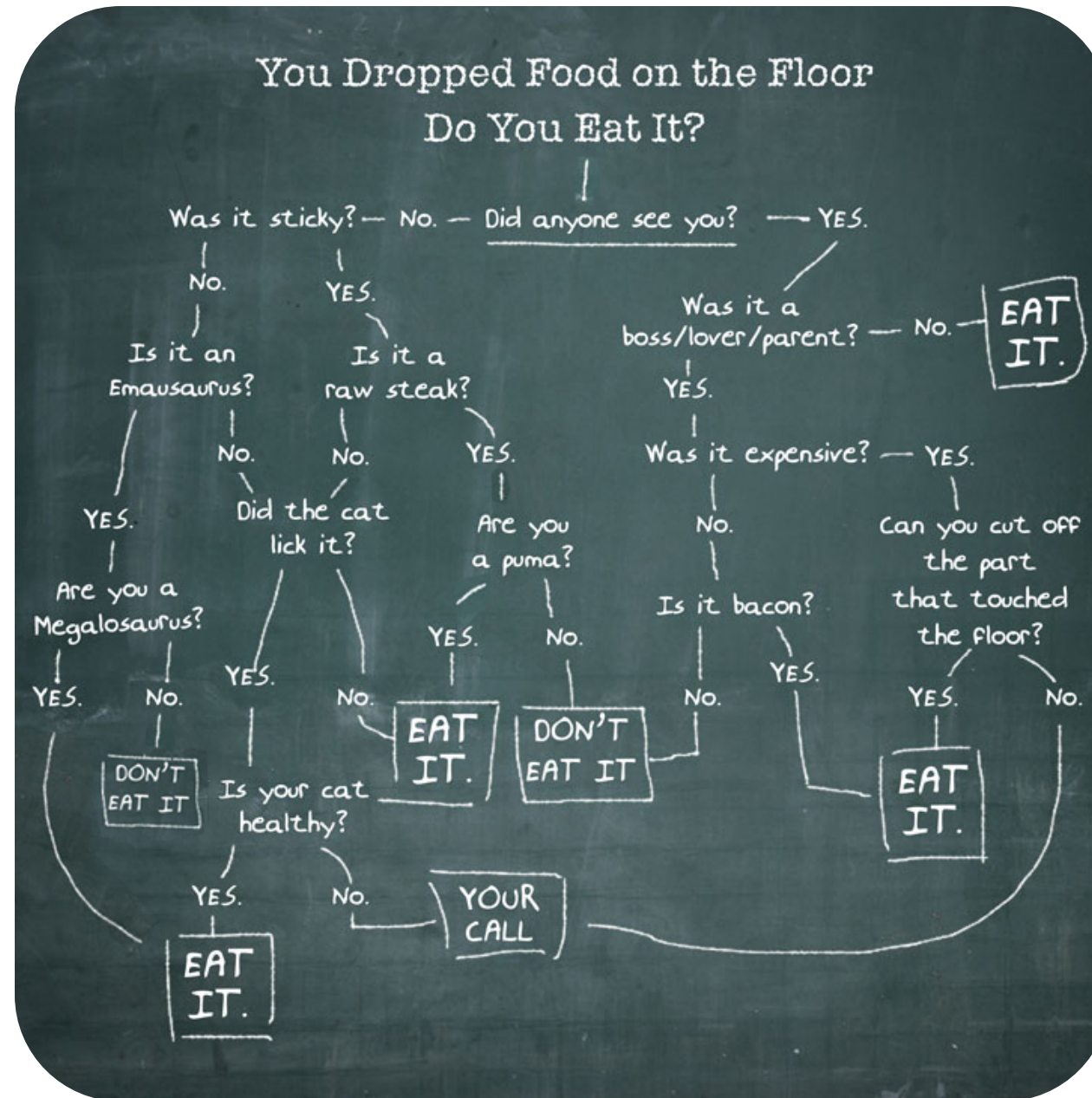


Today's Learning Objectives

Be able to answer:

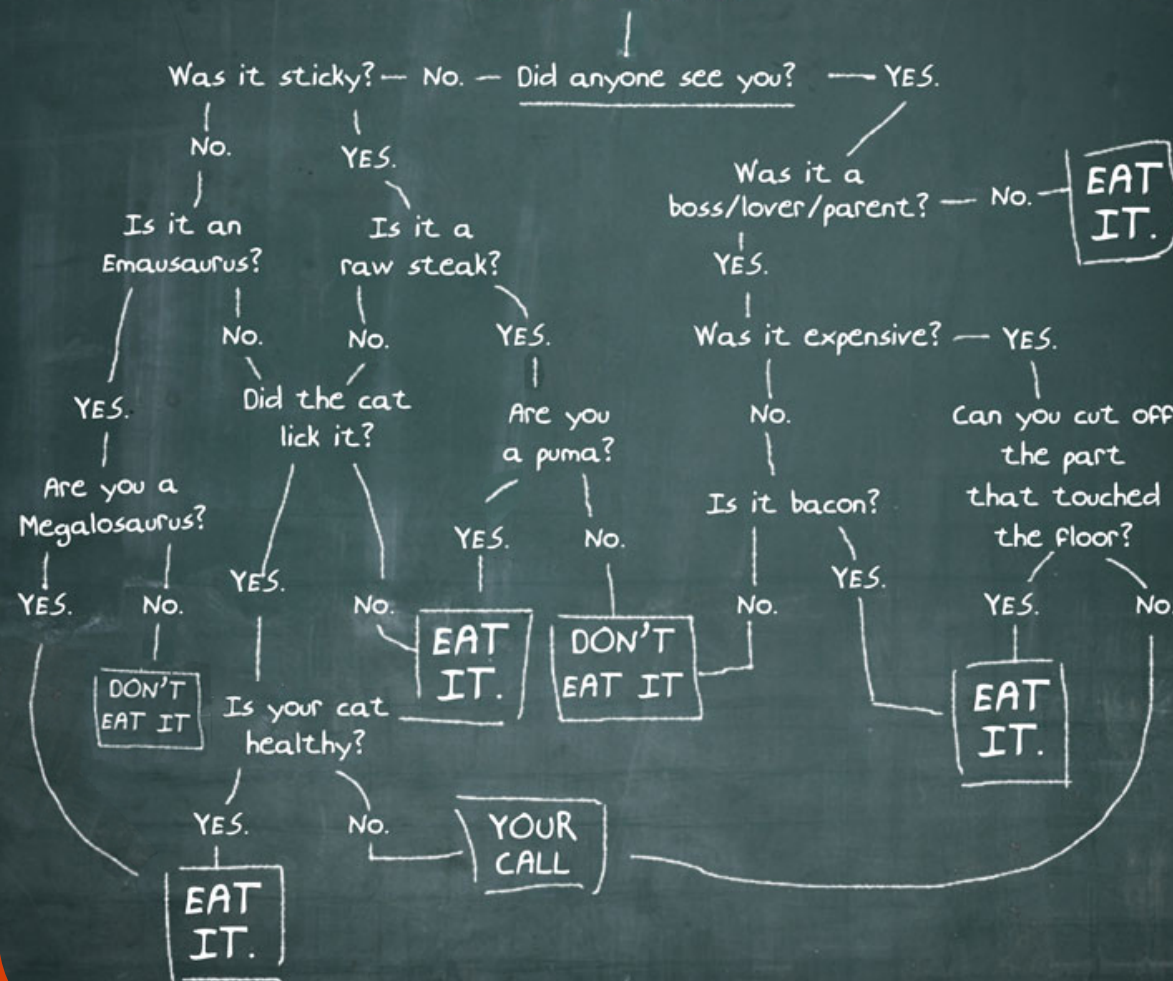
- ~~• What are the intuitions for some advanced structures in neural networks?~~
- What are decision trees?
- How are they learned?
 - What is entropy, conditional entropy, and information gain?
- How do they deal with continuous features?
- What do to about overfitting?







You Dropped Food on the Floor Do You Eat It?



Classify this new point according to this decision tree:

No one saw me drop my food.
The dropped food was sticky.
The dropped food is a steak.
I am not a puma.

Should I eat my dropped food?

B Eat it!

D Don't eat it!



Algorithm Name: Decision Trees

Type: Supervised Classifiers / Regressor

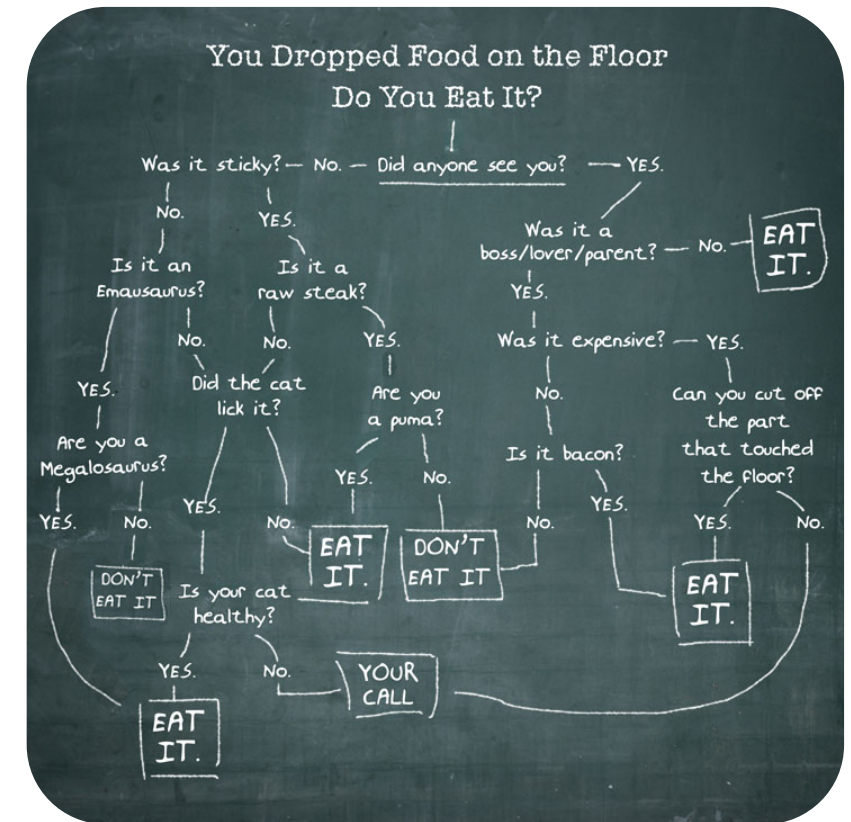
Synonyms: Classification and Regression Trees (CART)

Key Idea: Recursively subdivide the input space.
Works for discrete or continuous inputs.

Algorithms for learning decision trees:

- ID3
- C4.5 *CS*

Related concept: random forests (multiple decision trees)





Starting to Think About Decision Trees (Discrete Inputs)

Consider the dataset below - we want to predict fuel efficiency.

If you could only use one attribute to make the decision, which would you choose?



# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
6	Medium	Medium	Bad
4	Medium	Medium	Bad
8	High	Low	Bad
4	Low	Low	Good



Starting to Think About Decision Trees (Discrete Inputs)

When we split on a discrete attribute, it shatters our dataset into subsets for each attribute.

Split on # Cylinders

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Medium	Medium	Bad
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

Split on Weight

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad
4	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

Split on Acceleration

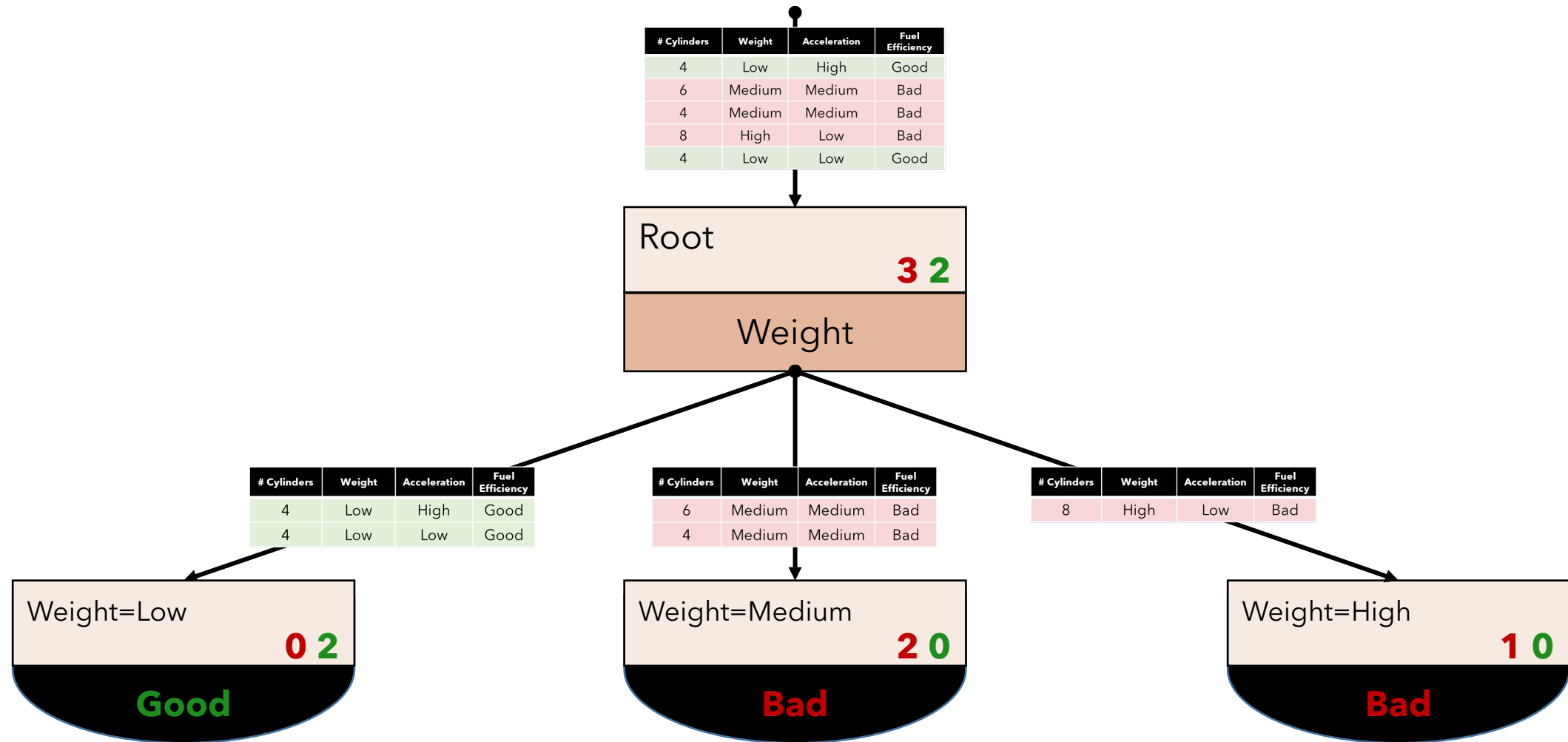
# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad
4	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good

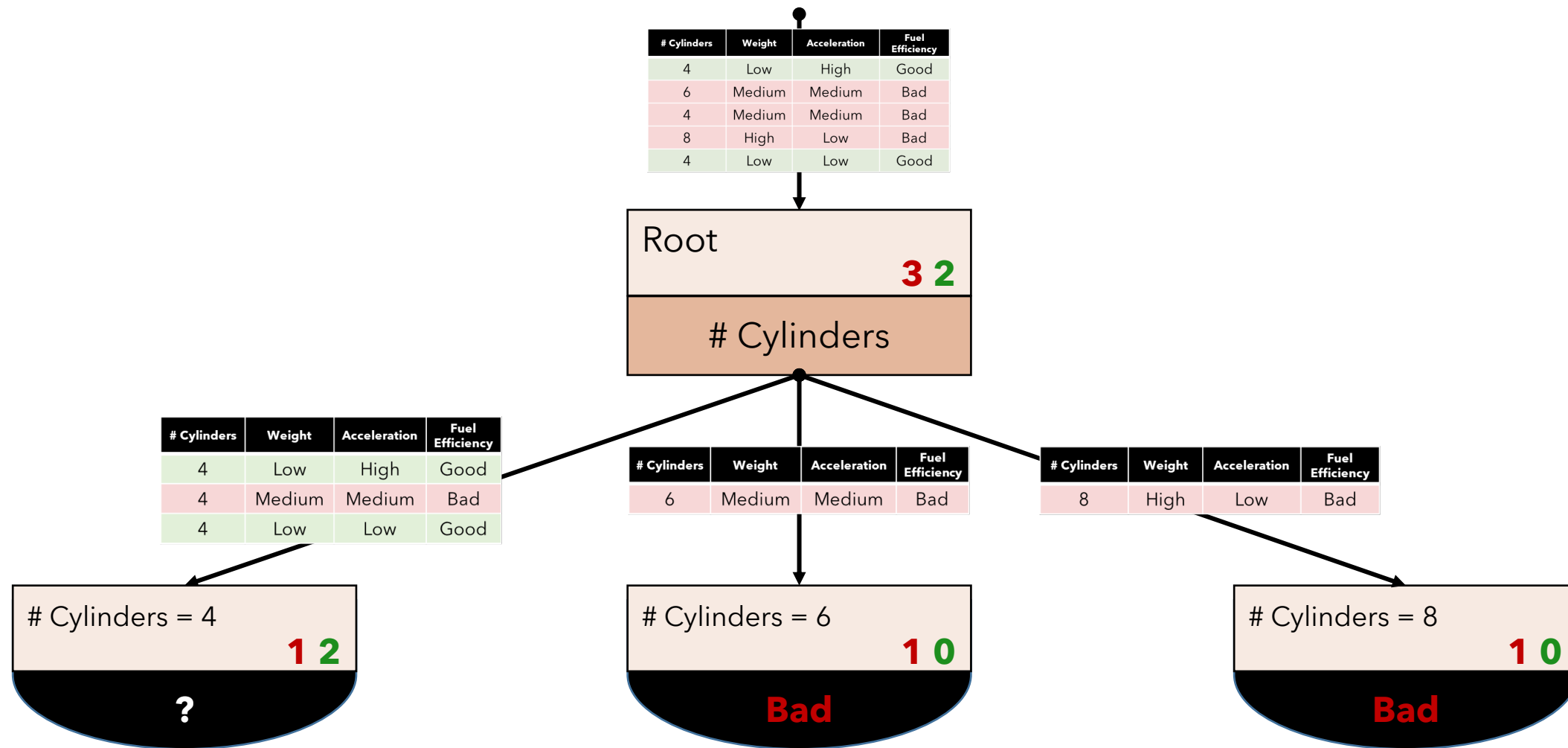


Starting to Think About Decision Trees (Discrete Inputs)





Starting to Think About Decision Trees (Discrete Inputs)

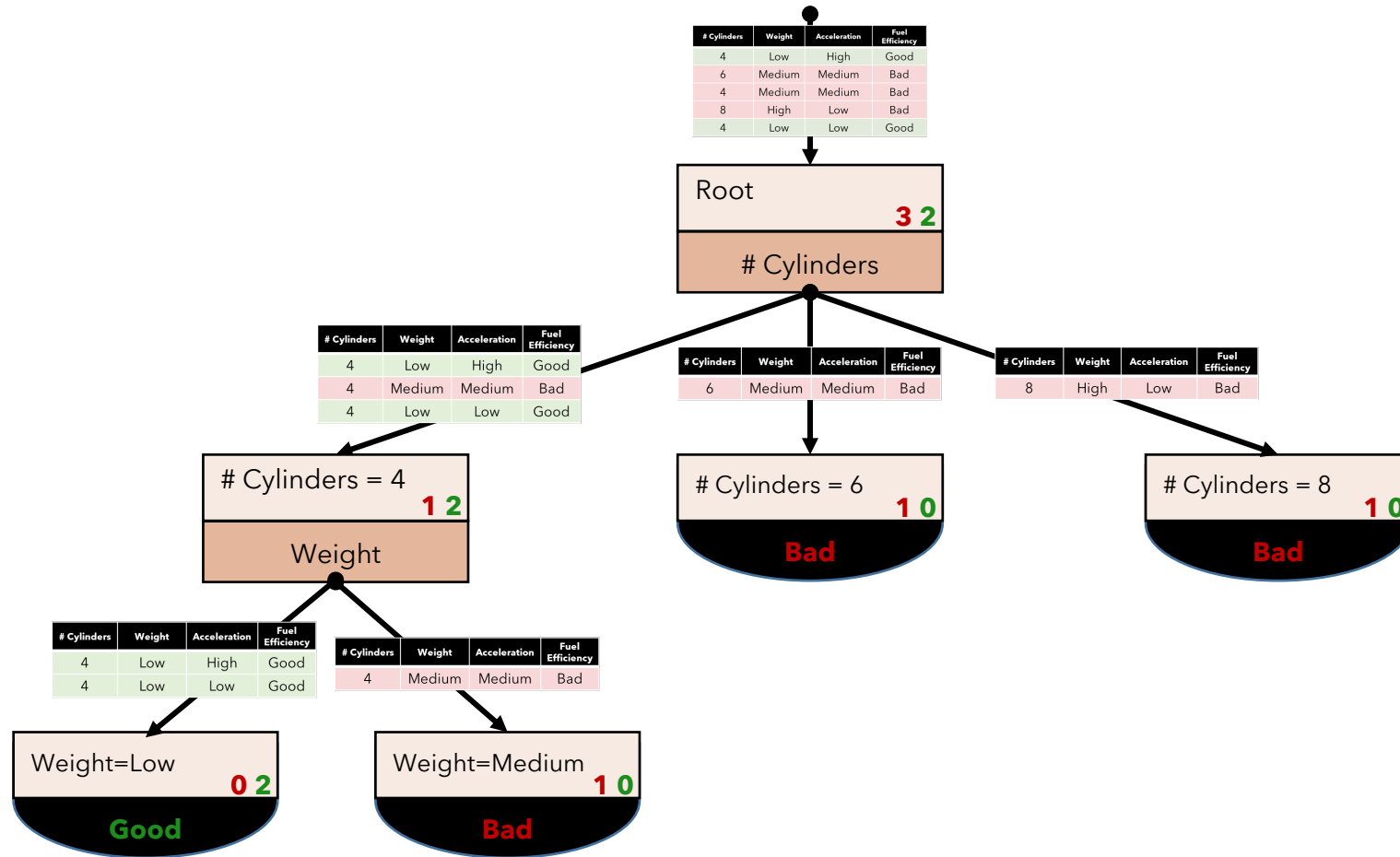


One of the splits isn't pure, what now?



Starting to Think About Decision Trees (Discrete Inputs)

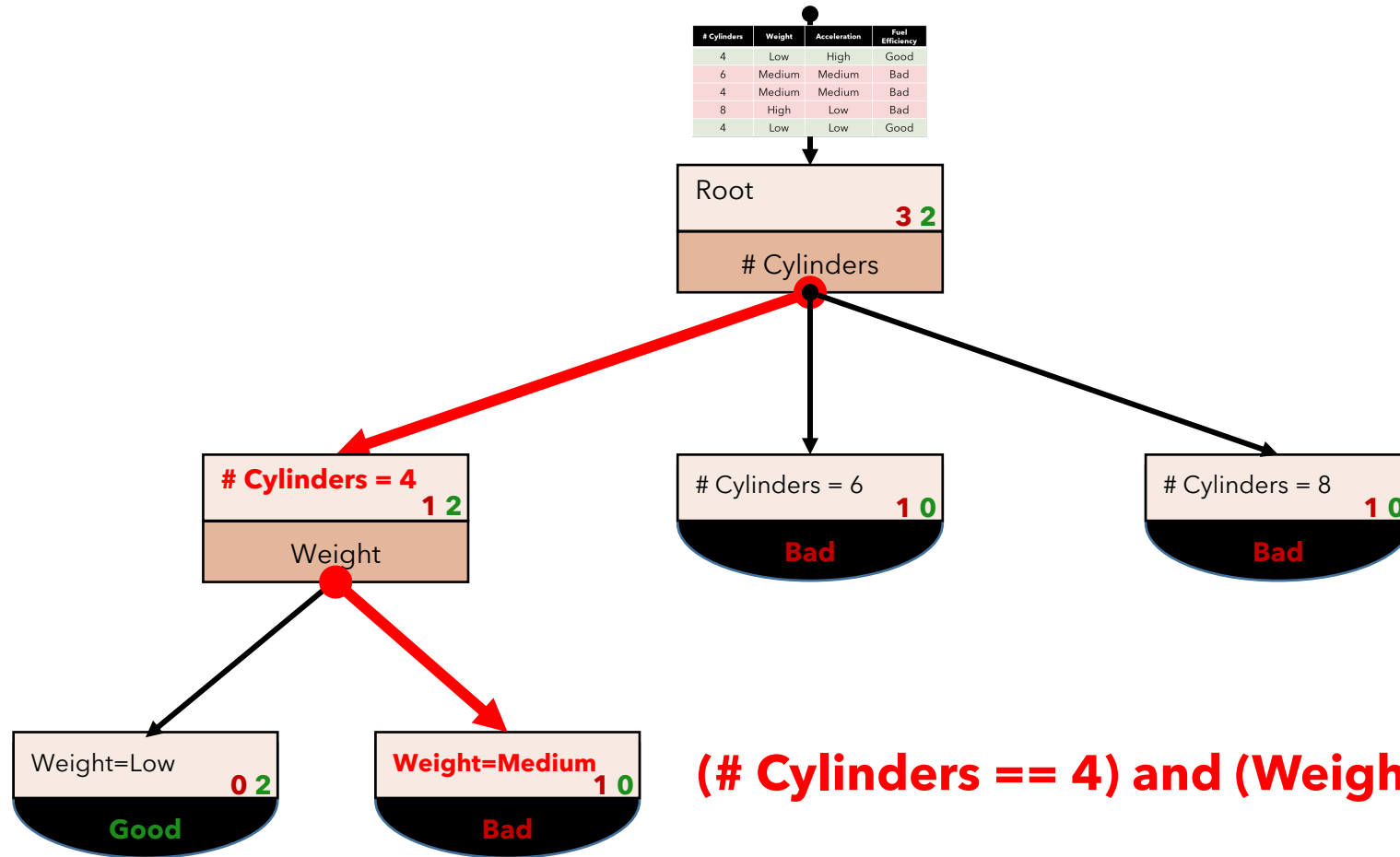
Split on another attribute and repeat this process..





Starting to Think About Decision Trees (Discrete Inputs)

Notice that paths in the tree are logical formulas:

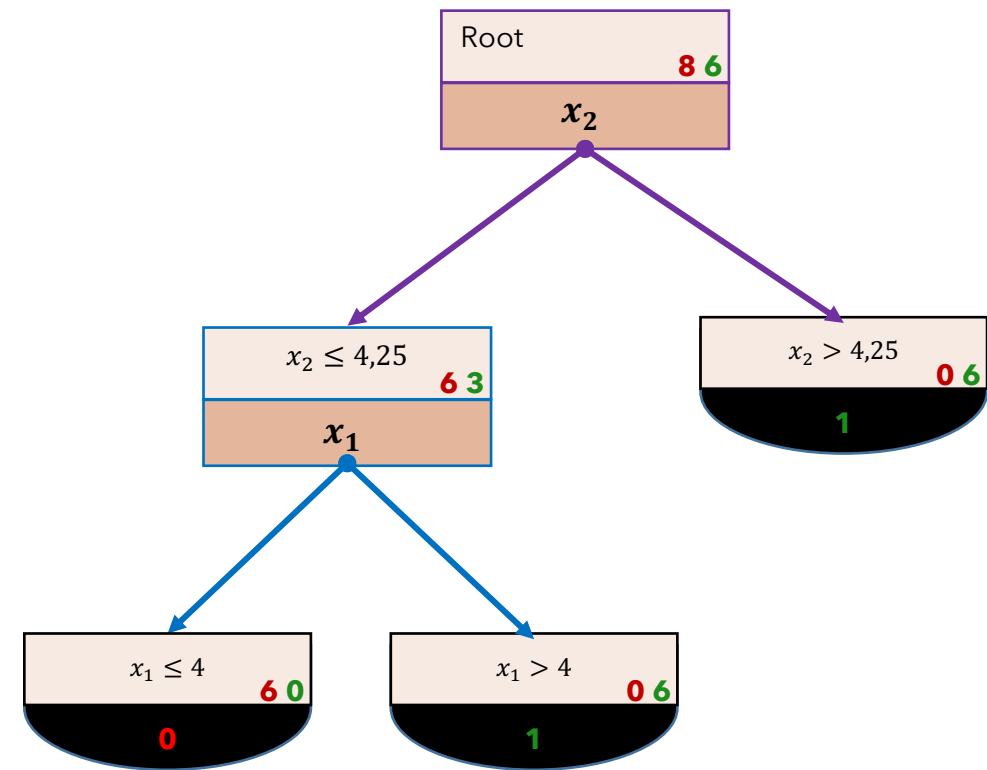
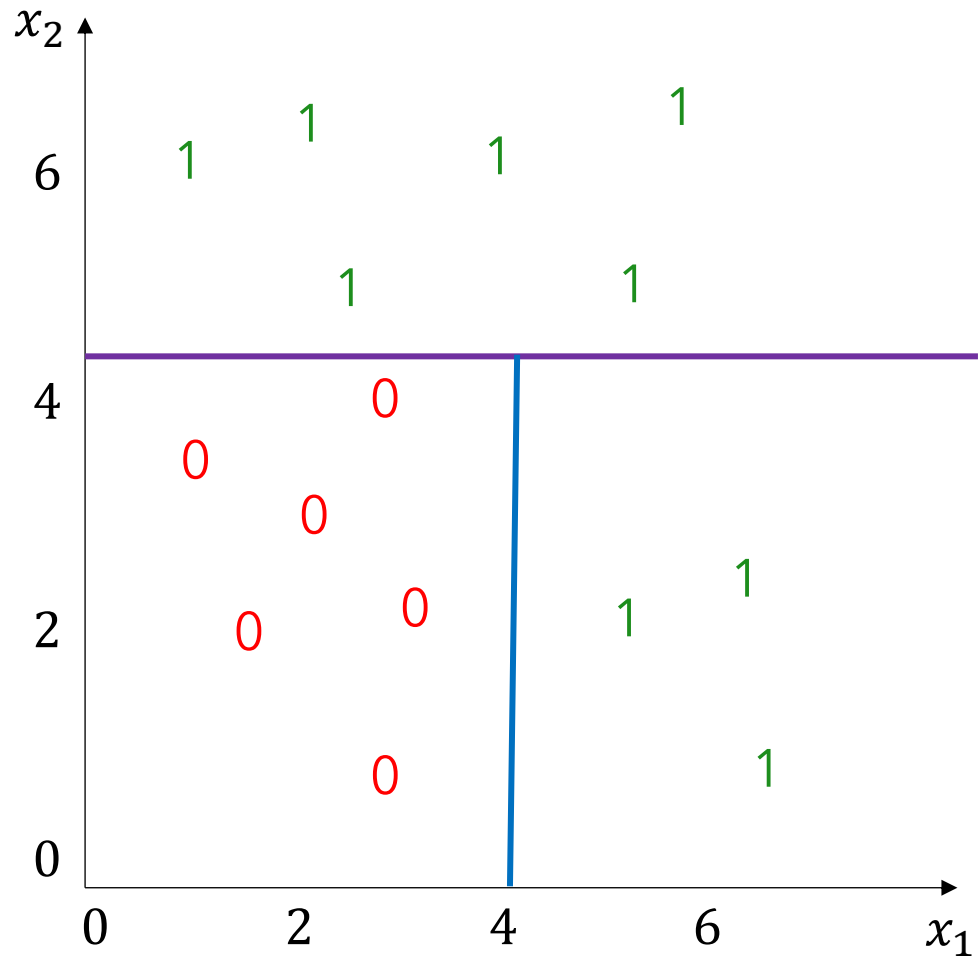


(# Cylinders == 4) and (Weight == Medium)



Starting to Think About Decision Trees (Continuous Inputs)

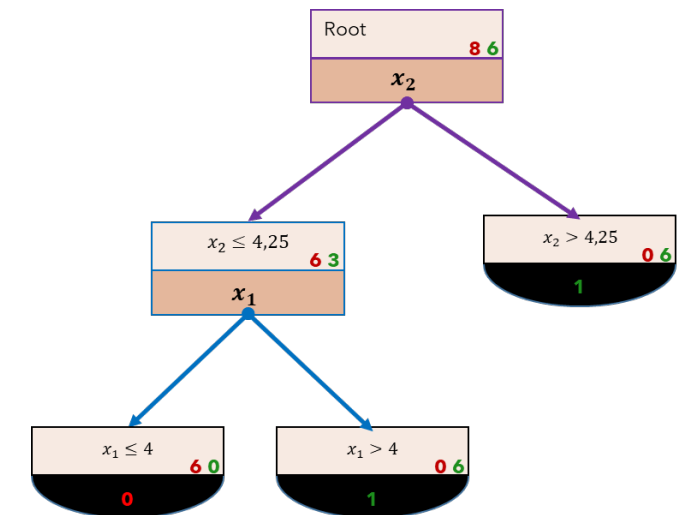
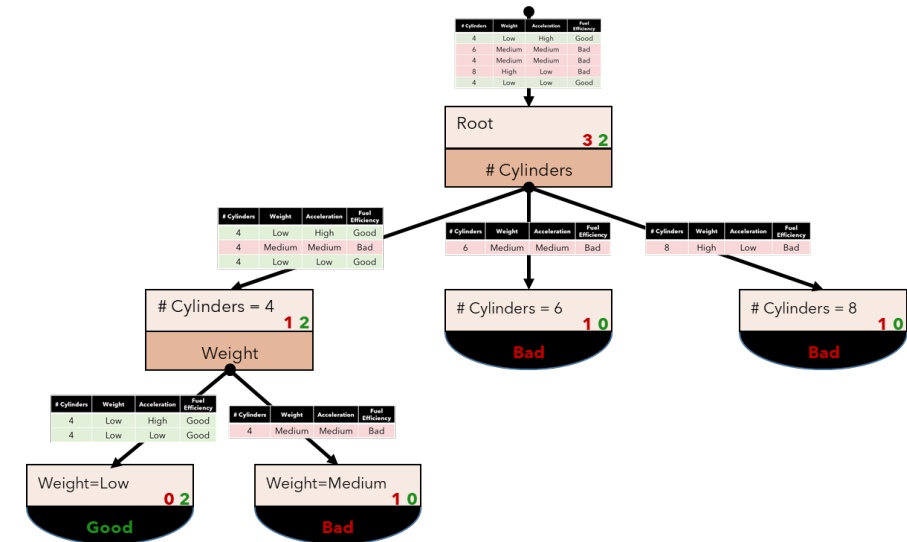
Decision trees divide the feature space into axis-parallel rectangles and label each rectangle with one class / value





A decision tree is a tree structured model where:

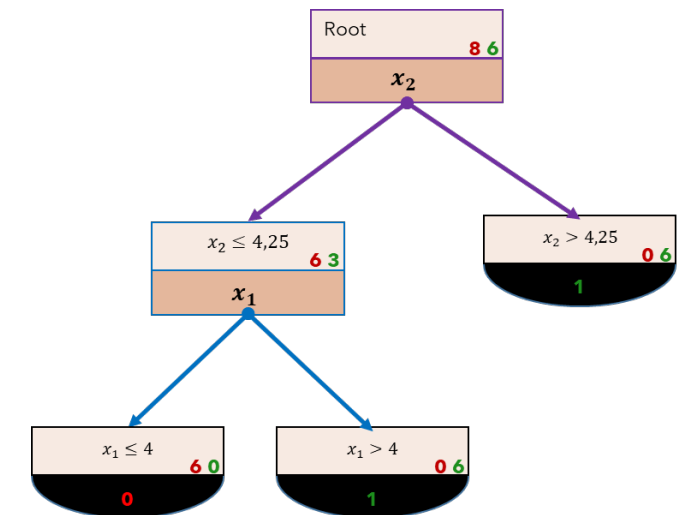
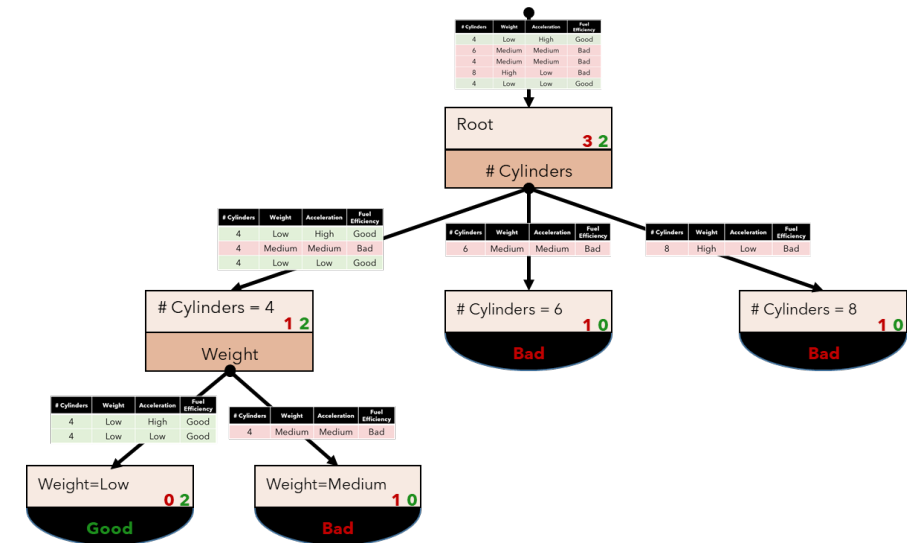
- **Internal nodes** perform tests against an individual input feature value
- Each branch from an internal node represents a potential value or range of values of the tested attribute.
- Each **leaf node** predicts an output
 - Either class or continuous value

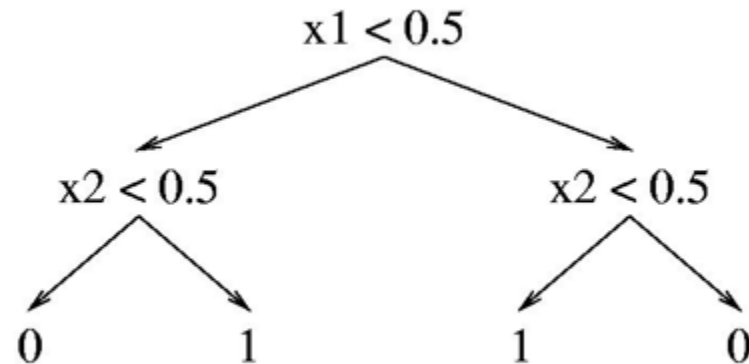
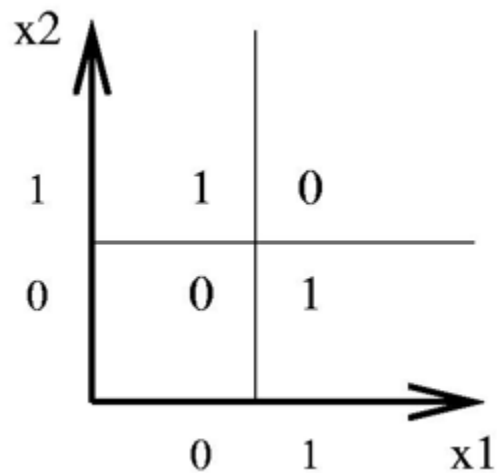




Decision trees have many appealing properties and are work-horses in many practical ML applications.

- Resulting models are human-interpretable (more or less)
- Deals with mixed discrete / continuous inputs without needing careful input encoding
 - Recall our binary encoding for categorical inputs in kNN for HW1
- Can represent arbitrarily complex functions as depth increases





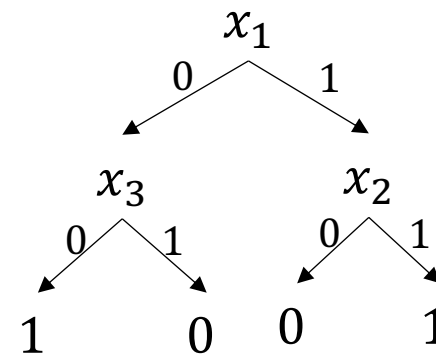
Decision trees can represent any arbitrary Boolean function.

- The tree will need to have exponentially many nodes in the worst case.



As the number of nodes (or depth) of the tree increases, the model can represent more complex functions (aka the hypothesis space grows):

- **Depth 1** ("decision stump") can represent any Boolean function of one feature.
- **Depth 2** Any Boolean function of two features and some Boolean functions of three features (e.g., $(x_1 \text{ and } x_2)$ or $(\text{not } x_1 \text{ and not } x_3)$)
- Etc..





A Real Example

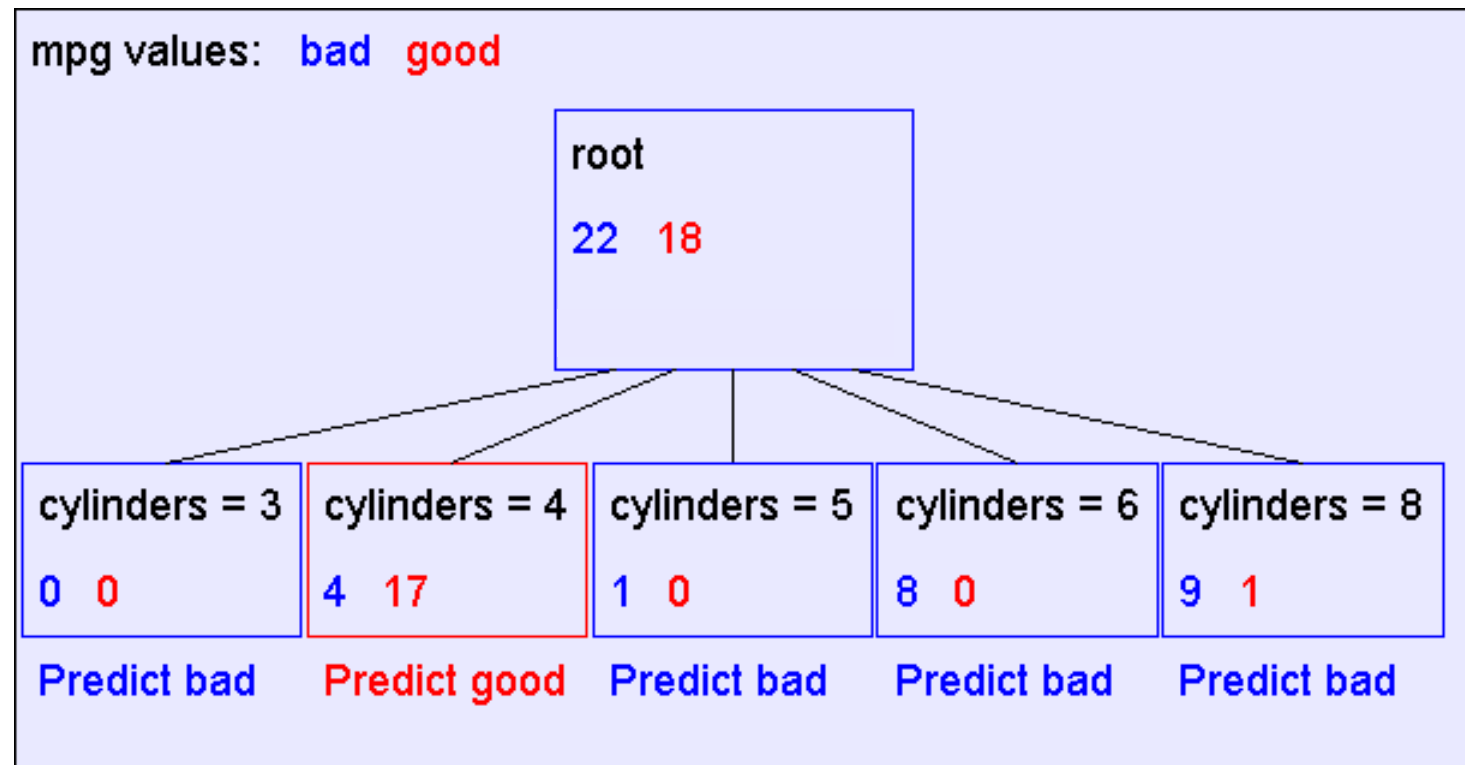
Dataset of car properties and mile per gallon as being good or bad. No continuous variables – some categorical, some ordinal. 40 records total.

mpg	cylinders	displacement	horsepower	weight	acceleration	modelyear	maker
good	4	low	low	low	high	75to78	asia
bad	6	medium	medium	medium	medium	70to74	america
bad	4	medium	medium	medium	low	75to78	europa
bad	8	high	high	high	low	70to74	america
bad	6	medium	medium	medium	medium	70to74	america
bad	4	low	medium	low	medium	70to74	asia
bad	4	low	medium	low	low	70to74	asia
bad	8	high	high	high	low	75to78	america
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
bad	8	high	high	high	low	70to74	america
good	8	high	medium	high	high	79to83	america
bad	8	high	high	high	low	75to78	america
good	4	low	low	low	low	79to83	america
bad	6	medium	medium	medium	high	75to78	america
good	4	medium	low	low	low	79to83	america
good	4	low	low	medium	high	79to83	america
bad	8	high	high	high	low	70to74	america
good	4	low	medium	low	medium	75to78	europa
bad	5	medium	medium	medium	medium	75to78	europa

<https://archive.ics.uci.edu/ml/datasets/auto+mpg>



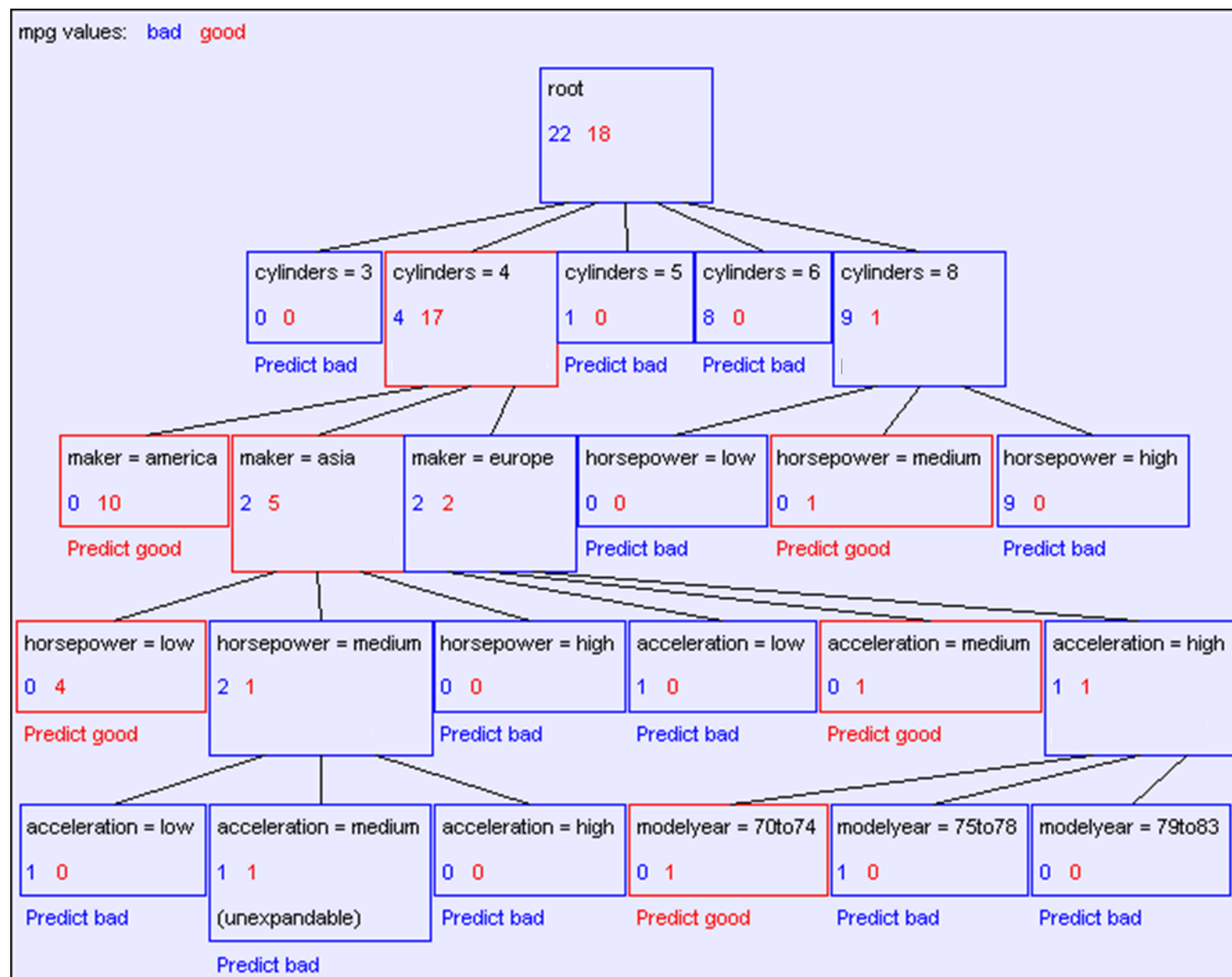
A Decision Stump – Splitting on number of cylinders





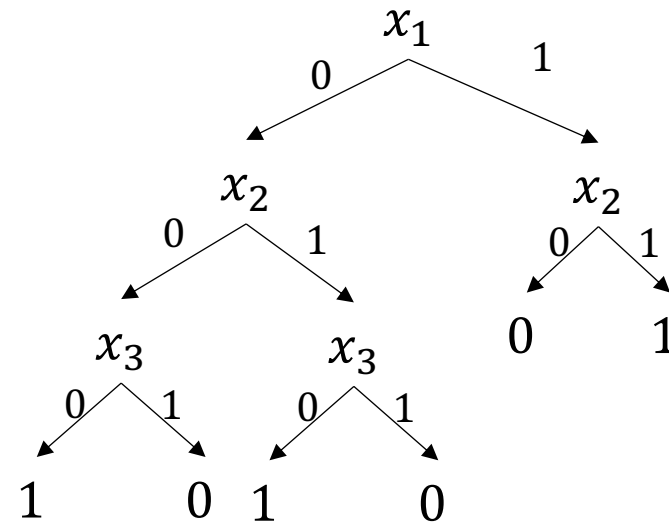
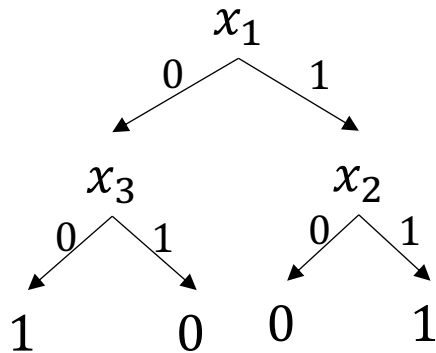
A Real Example

Full Tree





- Not all features/attributes need to appear in the tree
- A feature/attribute may appear in multiple branches
- On any given path, repeating discrete features is not useful
- Many trees represent the same concept / logical equation but not all trees will have the same size.





How do we learn decision trees from data?

Easy Mode: find a decision tree that achieves minimum error on training data. Trivially achievable with large enough tree

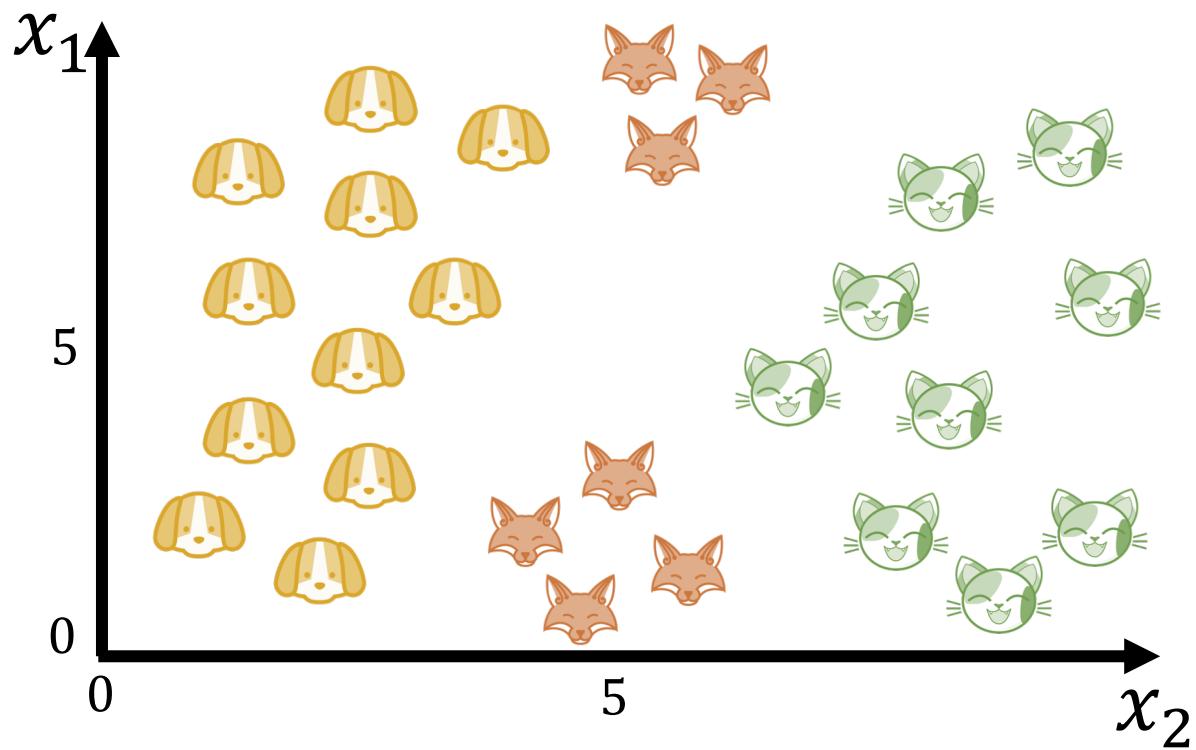
- Imagine a tree deep enough that each leaf contains only one example (or all examples sharing identical feature values)

NP-Hard Mode: find the **smallest** decision tree that achieves the minimum training error

- Why might we care about “smallest”?
Small trees represent *simpler* functions → less likely to overfit.

Most popular methods build the tree greedily – can’t guarantee finding smallest.

Let's Do It Intuitively/Manually First



Consider the simple recursive algorithm below:

BasicGreedyAlgorithm(dataset S):

Based on S , choose “best” test t to split the data

Split S into subset $S_1, \dots, S_k$ for each unique outcome of t applied to S

For i in $\{1, \dots, k\}$:

Create child node $c_i = \text{BasicGreedyAlgorithm}(S_i)$

Greedy Decision Tree Algorithm

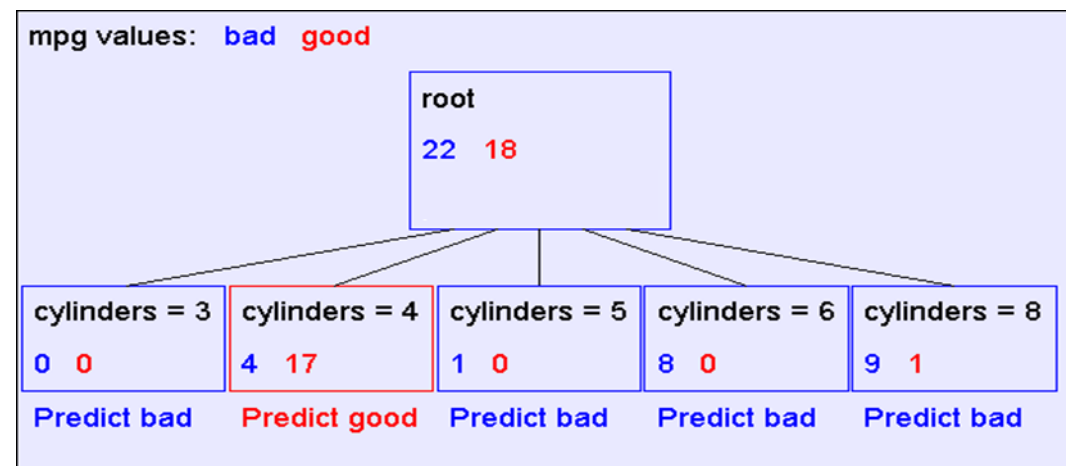
BasicGreedyAlgorithm(dataset S):

Based on S , choose “best” test t to split the data

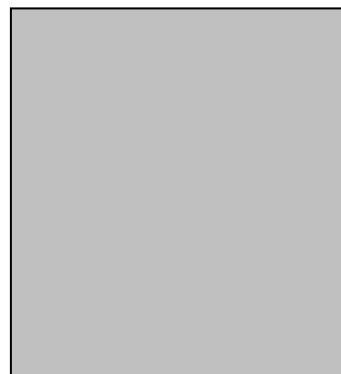
Split S into subset $S_1, \dots, S_k$ for each unique outcome of t applied to S

For i in $\{1, \dots, k\}$:

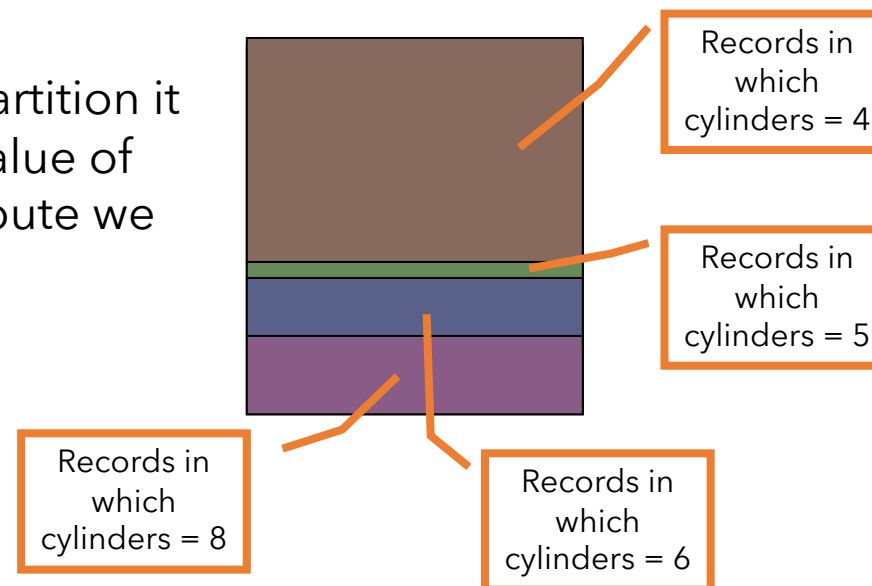
Create child node $c_i = \text{BasicGreedyAlgorithm}(S_i)$



Take the dataset...



...and partition it by the value of the attribute we split on



Greedy Decision Tree Algorithm

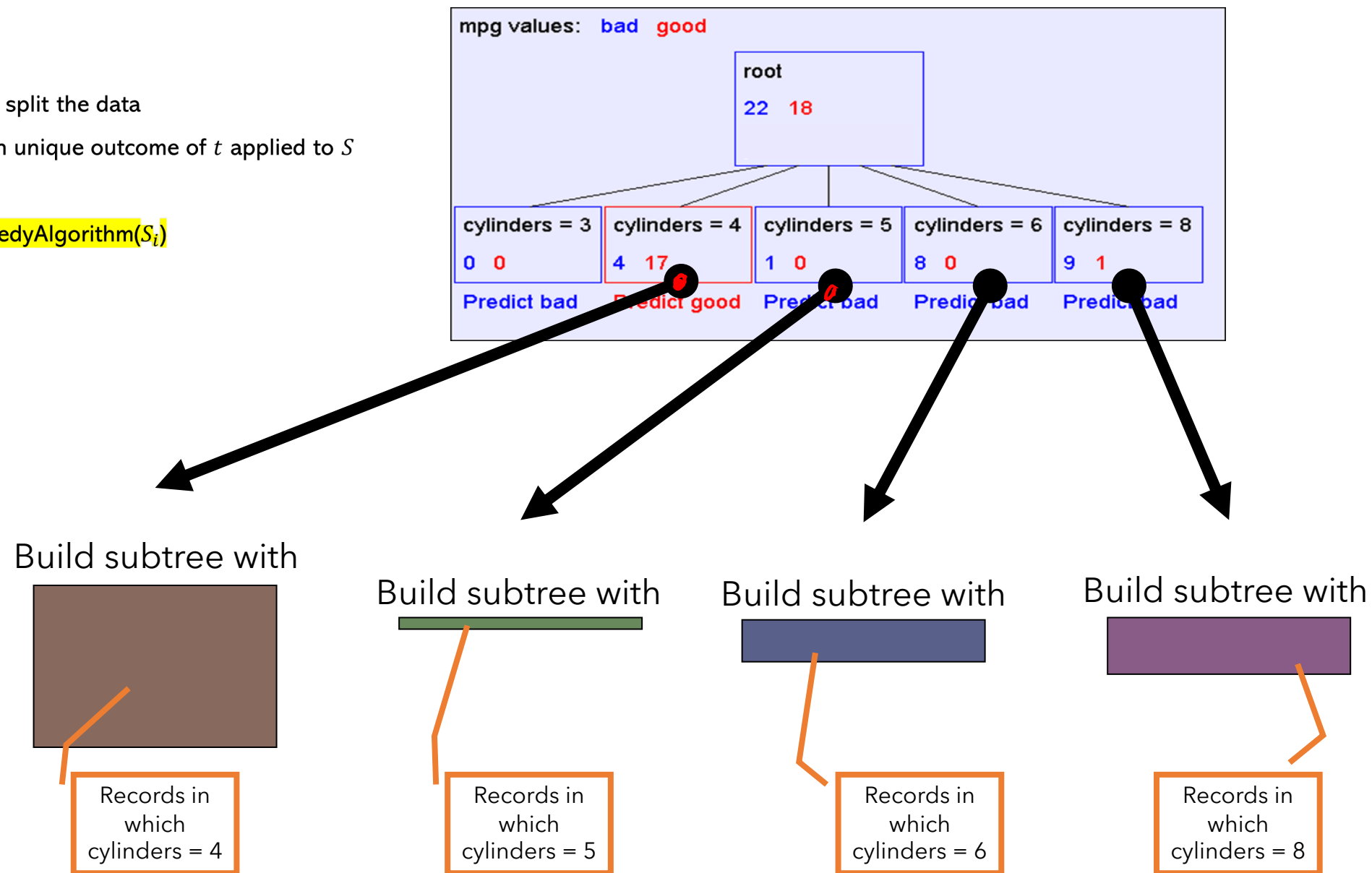
BasicGreedyAlgorithm(dataset S):

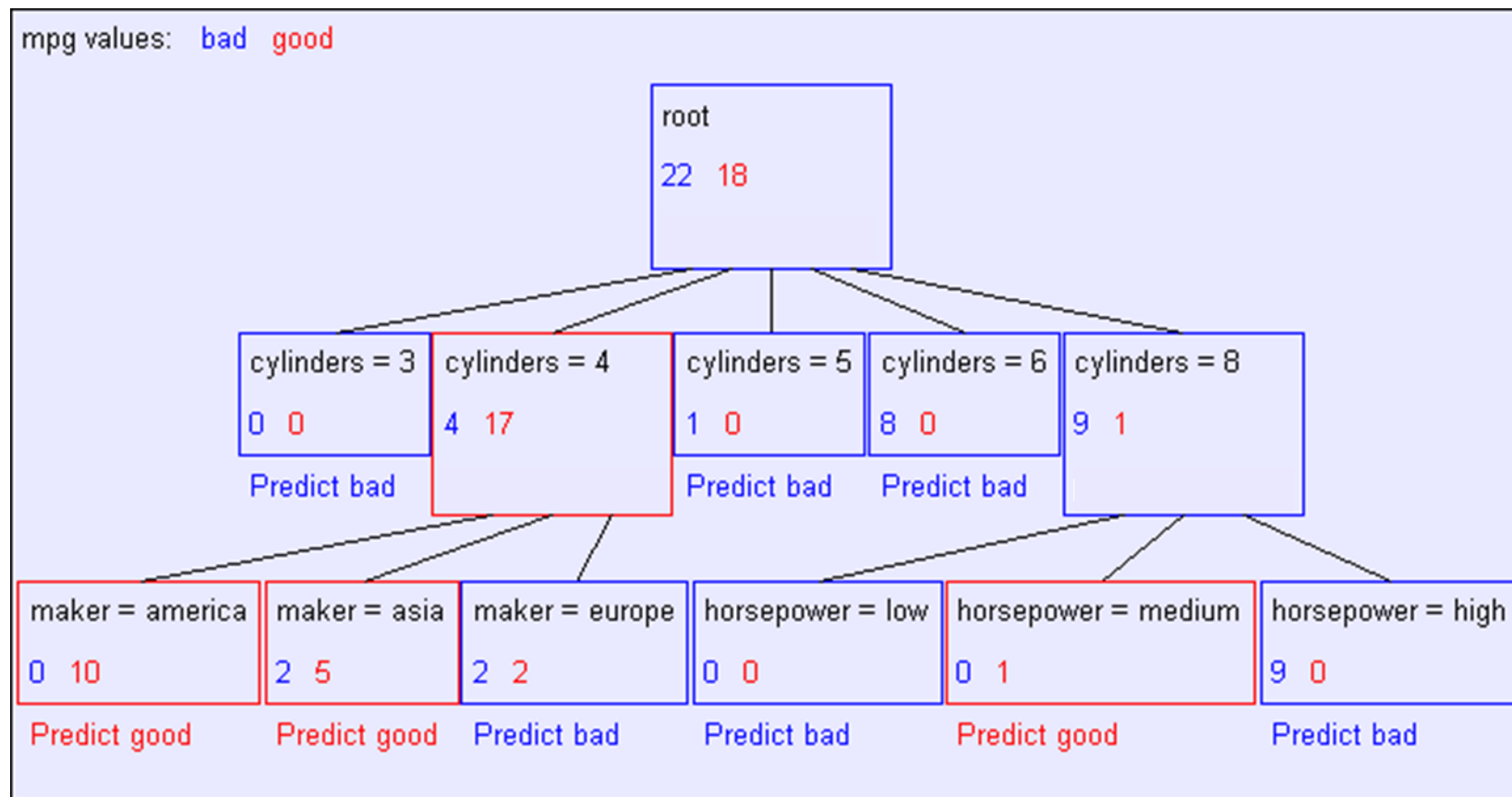
Based on S , choose “best” test t to split the data

Split S into subset $S_1, \dots, S_k$ for each unique outcome of t applied to S

For i in $\{1, \dots, k\}$:

Create child node $c_i = \text{BasicGreedyAlgorithm}(S_i)$





Back to our recursive learning algorithm:

BasicGreedyAlgorithm(dataset S):

Based on S , choose “best” test t to split the data

Split S into subset $S_1, \dots, S_k$ for each unique outcome of t applied to S

For i in $\{1, \dots, k\}$:

Create child node $c_i = \text{BasicGreedyAlgorithm}(S_i)$

**How do we know
which is best?**



When do we stop? What are the base cases of this recursion?



How to choose tests / splits / attributes?

One idea: Choose whichever test / attribute that reduces uncertainty about the class labels in the resulting dataset splits.

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
6	Medium	Medium	Bad
4	Medium	Medium	Bad
8	High	Low	Bad
4	Low	Low	Good

Split on # Cylinders

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Medium	Medium	Bad
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

Split on Weight

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad
4	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad



Entropy of a Random Variable Y : More uncertainty, more entropy!

Discrete Distributions:

$$H(Y) = - \sum_{i=1}^k P(Y = y_i) \log_2 P(Y = y_i)$$

Continuous Distributions:

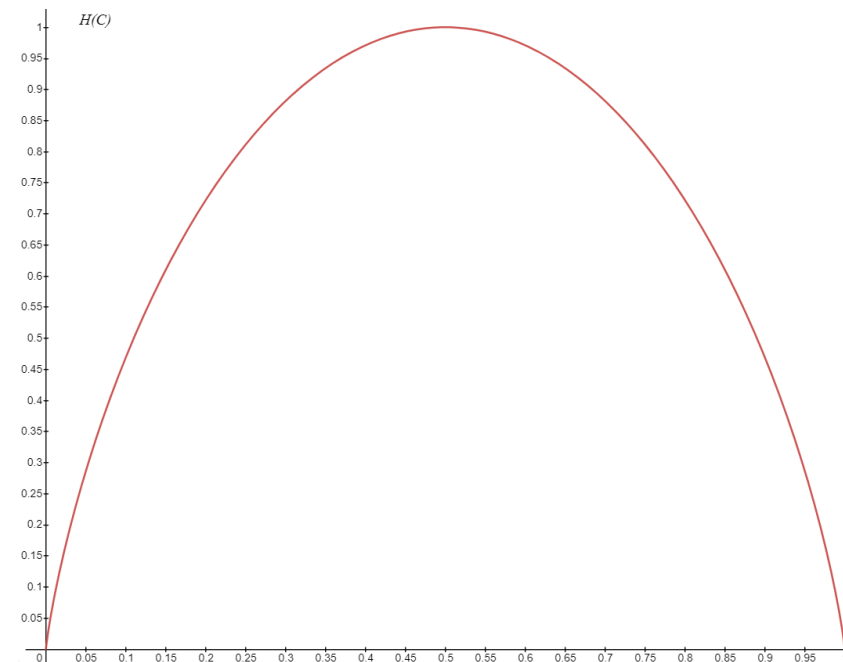
$$H(Y) = - \int_{y \in Y} P(Y = y) \log_2 P(Y = y) dy$$



For example, the entropy of the outcome of a coin flip C with probability of heads θ is:

$$\begin{aligned} H(C) &= - \sum_{i=1}^k P(Y = y_i) \log_2 P(Y = y_i) \\ &= -\theta \log_2 \theta - (1 - \theta) \log_2(1 - \theta) \end{aligned}$$

The flip is most uncertain at $\theta = 0.5$ and least at either $\theta = 0$ or $\theta = 1$ where the outcome is fixed.





$$H(Y) = - \sum_{i=1}^k P(Y = y_i) \log_2 P(Y = y_i)$$

What about the entropy of the class labels in a dataset?

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
6	Medium	Medium	Bad
4	Medium	Medium	Bad
8	High	Low	Bad
4	Low	Low	Good

$$P(Y = \textit{Good}) = 2/5$$

$$P(Y = \textit{Bad}) = 3/5$$

$$H(Y) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \approx 0.971$$



How to choose tests / splits / attributes?

One idea: Choose whichever test / attribute that reduces uncertainty about the class labels in the resulting dataset splits.

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
6	Medium	Medium	Bad
4	Medium	Medium	Bad
8	High	Low	Bad
4	Low	Low	Good

$$H(Y) \approx 0.971$$

Split on # Cylinders

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Medium	Medium	Bad
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

$$H(Y|Cylinders) \approx ?$$

Split on Weight

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad
4	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

$$H(Y|Weight) \approx ?$$



Conditional Entropy of a Random Variable Y Given X :

Entropy of Y when only
considering records where $X = x_j$

$$H(Y|X) = - \sum_j^v P(X = x_j) \sum_{i=1}^k P(Y = y_i | X = x_j) \log_2 P(Y = y_i | X = x_j)$$

Weighted by
probability of $X = x_j$

Continuous distribution version replaces sums with integrals as is typical.



What is the conditional entropy of the class labels in the following split?

Split on # Cylinders

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Medium	Medium	Bad
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

$$P(Y = \text{Good} \mid \#Cylinders = 4) = 2/3$$

$$P(Y = \text{Bad} \mid \#Cylinders = 4) = 1/3$$

$$P(\#Cylinders = 4) = 3/5$$

$$P(Y = \text{Good} \mid \#Cylinders = 6) = 0$$

$$P(Y = \text{Bad} \mid \#Cylinders = 6) = 1$$

$$P(\#Cylinders = 6) = 1/5$$

$$P(Y = \text{Good} \mid \#Cylinders = 8) = 0$$

$$P(Y = \text{Bad} \mid \#Cylinders = 8) = 1$$

$$P(\#Cylinders = 8) = 1/5$$

$$H(Y|X) = -\frac{3}{5} \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) - \frac{1}{5} \left(\frac{0}{1} \log_2 \frac{0}{1} + 1 \log_2 1 \right) - \frac{1}{5} \left(\frac{0}{1} \log_2 \frac{0}{1} + 1 \log_2 1 \right)$$

$$\approx 0.551$$



Split on Weight

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad
4	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

What is the conditional entropy of this data split?

$$H(Y|Weight) \approx ?$$

A 1

B 0

C 0.579

D 0.286



How to choose tests / splits / attributes?

One idea: Choose whichever test / attribute that reduces uncertainty about the class labels in the resulting dataset splits.

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
6	Medium	Medium	Bad
4	Medium	Medium	Bad
8	High	Low	Bad
4	Low	Low	Good

$$H(Y) \approx 0.971$$

Split on # Cylinders

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Medium	Medium	Bad
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

$$H(Y|\text{Cylinders}) \approx 0.551$$

Split on Weight

# Cylinders	Weight	Acceleration	Fuel Efficiency
4	Low	High	Good
4	Low	Low	Good

# Cylinders	Weight	Acceleration	Fuel Efficiency
6	Medium	Medium	Bad
4	Medium	Medium	Bad

# Cylinders	Weight	Acceleration	Fuel Efficiency
8	High	Low	Bad

$$H(Y|\text{Weight}) \approx 0$$

Information Gain: $IG(X) = H(Y) - H(Y|X)$

Entropy of class label before splitting on an attribute minus the conditional entropy after the split.



$$IG(\#Cylinders) = H(Y) - H(Y|\#Cylinders) = 0.971 - 0.551 = 0.42$$

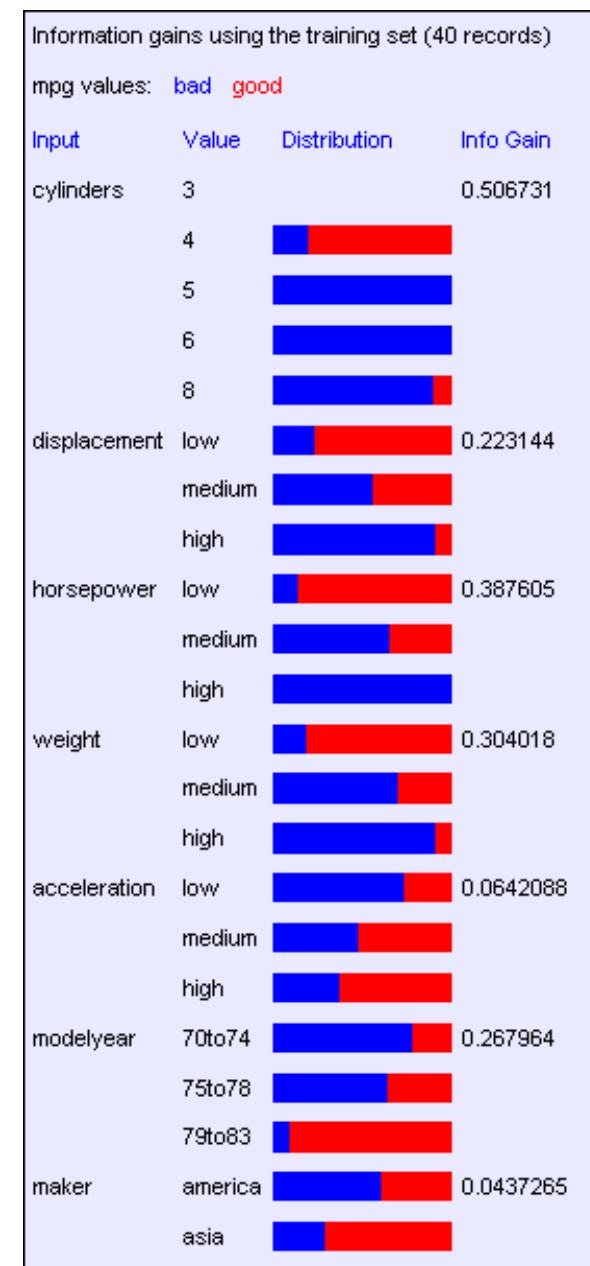
$$IG(Weight) = H(Y) - H(Y|Weight) = 0.971 - 0 = 0.971$$

We reduce uncertainty *more* by choosing to split on the Weight attribute.

Information Gain for the every first split

mpg	cylinders	displacement	horsepower	weight	acceleration	modelyear	maker
good	4	low	low	low	high	75to78	asia
bad	6	medium	medium	medium	medium	70to74	america
bad	4	medium	medium	medium	low	75to78	europa
bad	8	high	high	high	low	70to74	america
bad	6	medium	medium	medium	medium	70to74	america
bad	4	low	medium	low	medium	70to74	asia
bad	4	low	medium	low	low	70to74	asia
bad	8	high	high	high	low	75to78	america
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:
bad	8	high	high	high	low	70to74	america
good	8	high	medium	high	high	79to83	america
bad	8	high	high	high	low	75to78	america
good	4	low	low	low	low	79to83	america
bad	6	medium	medium	medium	high	75to78	america
good	4	medium	low	low	low	79to83	america
good	4	low	low	medium	high	79to83	america
bad	8	high	high	high	low	70to74	america
good	4	low	medium	low	medium	75to78	europa
bad	5	medium	medium	medium	medium	75to78	europa

<https://archive.ics.uci.edu/ml/datasets/auto+mpg>



Back to our recursive learning algorithm:

BasicGreedyAlgorithm(dataset S):

Based on S , choose “best” test t to split the data:

→ Evaluate the information gain of possible splits and pick the largest

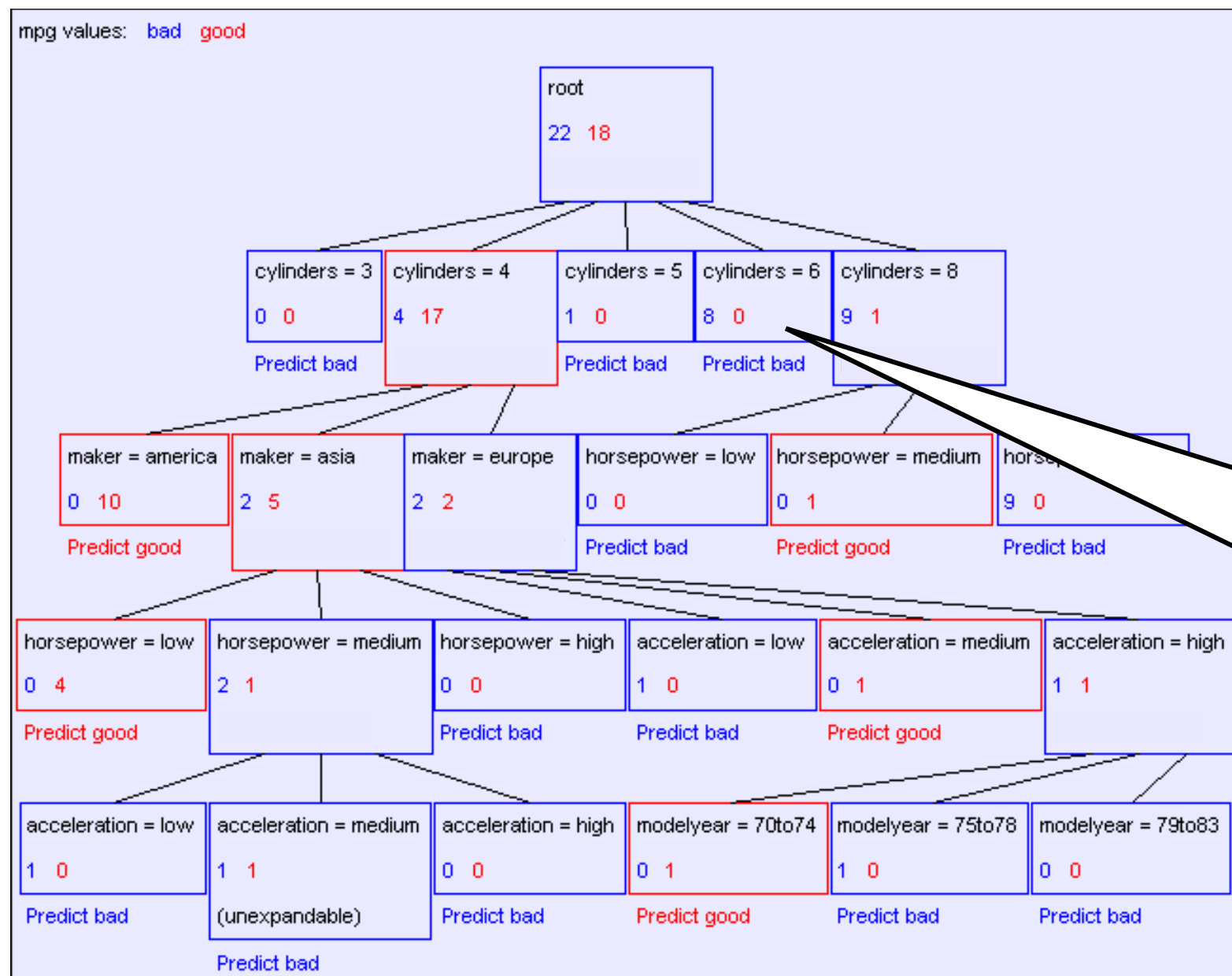
Split S into subset $S_1, \dots, S_k$ for each unique outcome of t applied to S

For i in $\{1, \dots, k\}$:

Create child node $c_i = \text{BasicGreedyAlgorithm}(S_i)$

When do we stop? What are the base cases of this recursion?

When do we stop? Base Case 1



Don't split a node if all matching records have the same output value

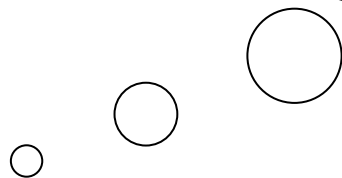


When do we stop?

- **Base Case One:** If all records in current data subset have the same output/class then **don't recurse because no further improvement could be made**
- **Base Case Two:** If all records have exactly the same set of input features/attributes then **don't recurse because nothing will split them**

An Intuitive Sounding Base Case 3:

If all attributes have zero information gain, then **don't recurse**





The Problem With Stopping At 0 Information Gain

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Suppose that **$Y = A \text{ XOR } B$**

What happens if we use the proposed Base Case 3?

Split on A

A	B	Y
0	0	0
0	1	1

A	B	Y
1	0	1
1	1	0

Split on B

A	B	Y
0	0	0
1	0	1

A	B	Y
0	1	1
1	1	0



The Problem With Stopping At 0 Information Gain

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

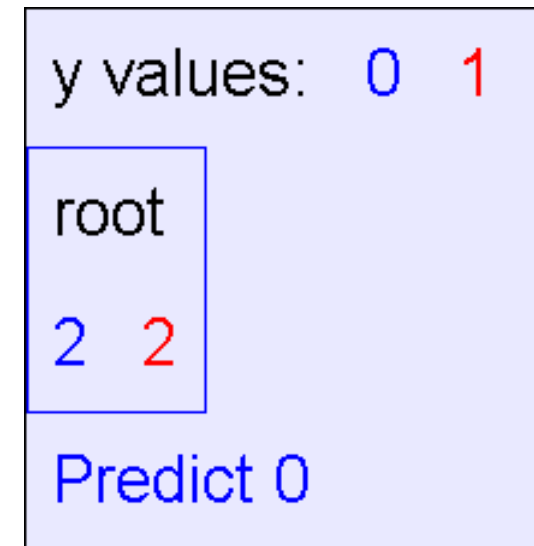
Suppose that **$Y = A \text{ XOR } B$**

What happens if we use the proposed Base Case 3?

The information gains:

Information gains using the training set (4 records)			
y values: 0 1			
Input	Value	Distribution	Info Gain
a	0		0
	1		
b	0		0
	1		

The resulting decision tree:



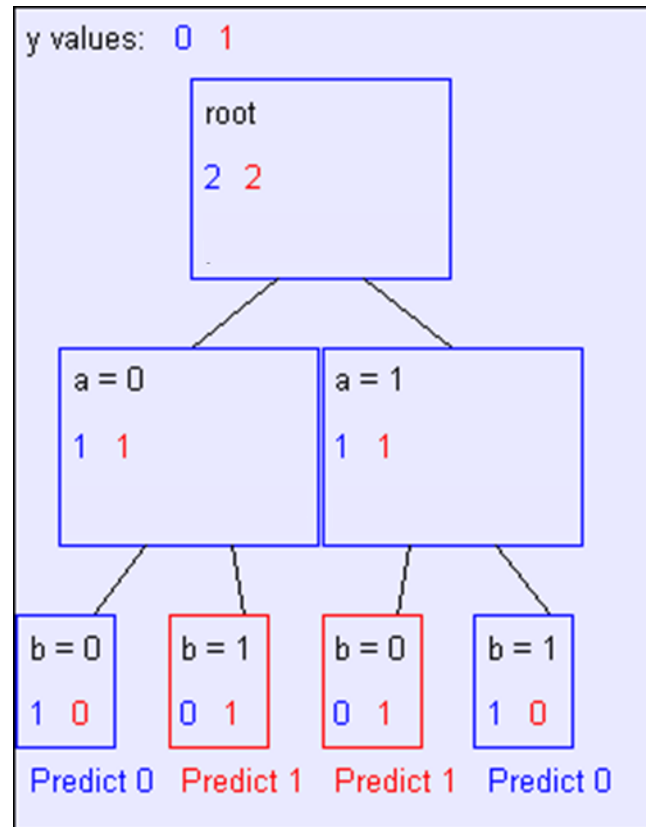


The Problem With Stopping At 0 Information Gain

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Suppose that **$Y = A \text{ XOR } B$**

If we ignore the proposed (bad idea) base case:



Back to our recursive learning algorithm:

BasicGreedyAlgorithm(dataset S):

If S all have same label or all same features:

return Leaf(majorityLabel(S))

Based on S , choose “best” test t to split the data:

Evaluate the information gain of possible splits and pick the largest

Split S into subset $S_1, \dots, S_k$ for each unique outcome of t applied to S

For i in $\{1, \dots, k\}$:

Create child node $c_i = \text{BasicGreedyAlgorithm}(S_i)$

Want to make splits of the form $X_i \geq t$ for some threshold t :

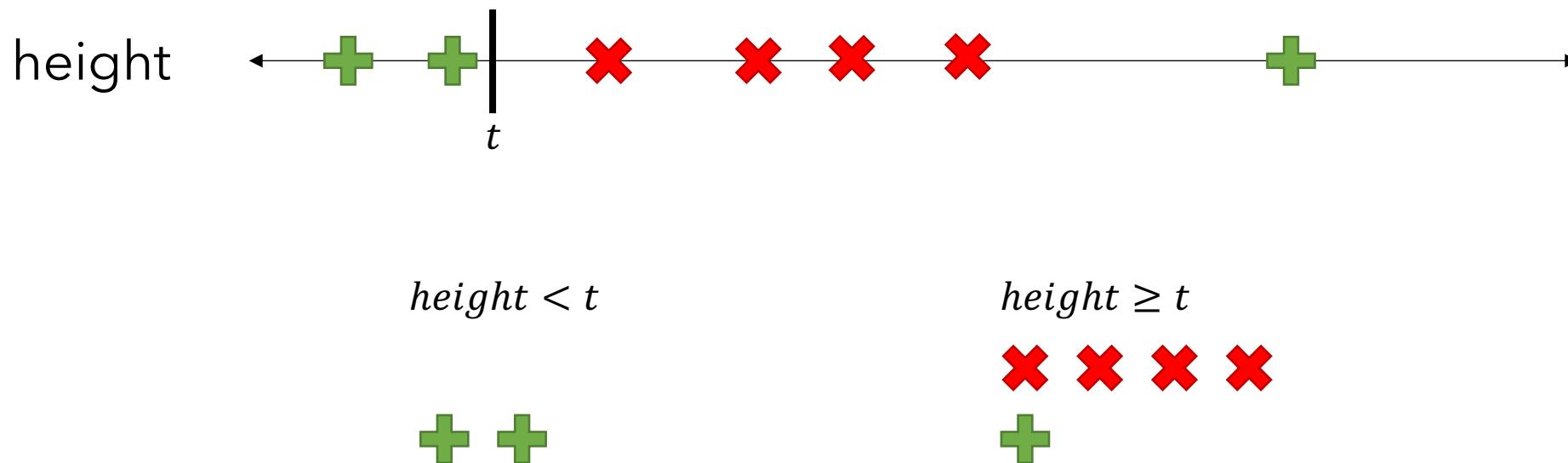
- Now need to pick the best attribute (X_i) and a threshold t
- There are infinitely many possible such splits... can't compute information gain for all of them!



- Any ideas?

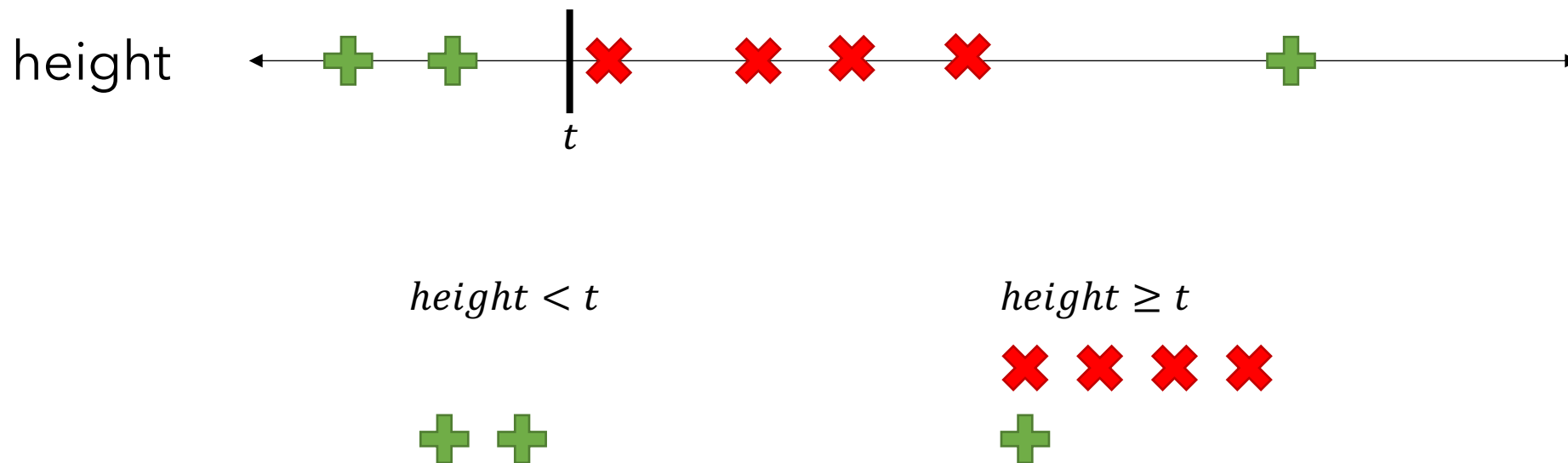
Want to make splits of the form $X_i \geq t$ for some threshold t :

- There are infinitely many possible such splits, but the information gain only changes when a threshold crosses at datapoint



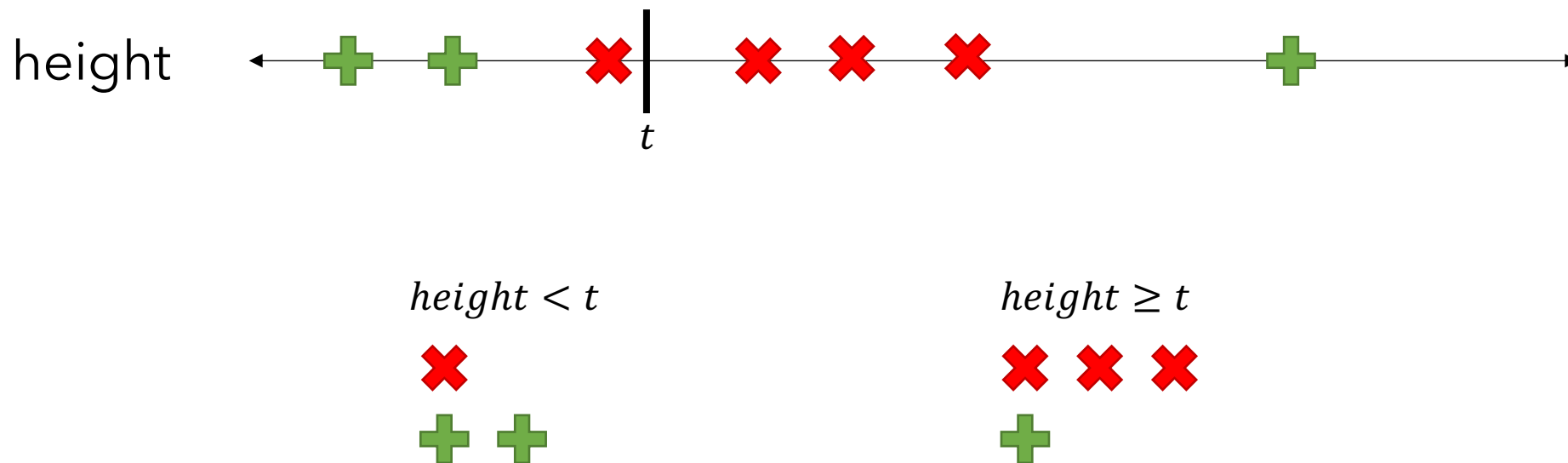
Want to make splits of the form $X_i \geq t$ for some threshold t :

- There are infinitely many possible such splits, but the information gain only changes when a threshold crosses at datapoint



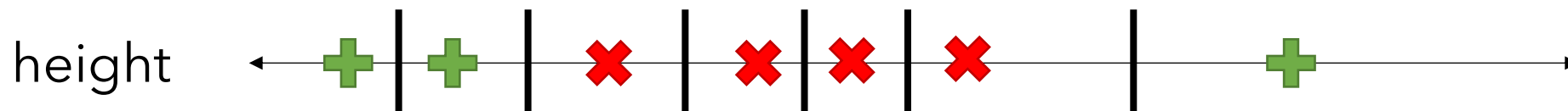
Want to make splits of the form $X_i \geq t$ for some threshold t :

- There are infinitely many possible such splits, but the information gain only changes when a threshold crosses at datapoint



Want to make splits of the form $X_i \geq t$ for some threshold t :

- Sort the values of X_i in the dataset, consider thresholds that occur in between consecutive datapoints.
- Compute information gain for each and choose the max.





Considering both discrete and continuous features

Suppose we have both continuous and discrete features:

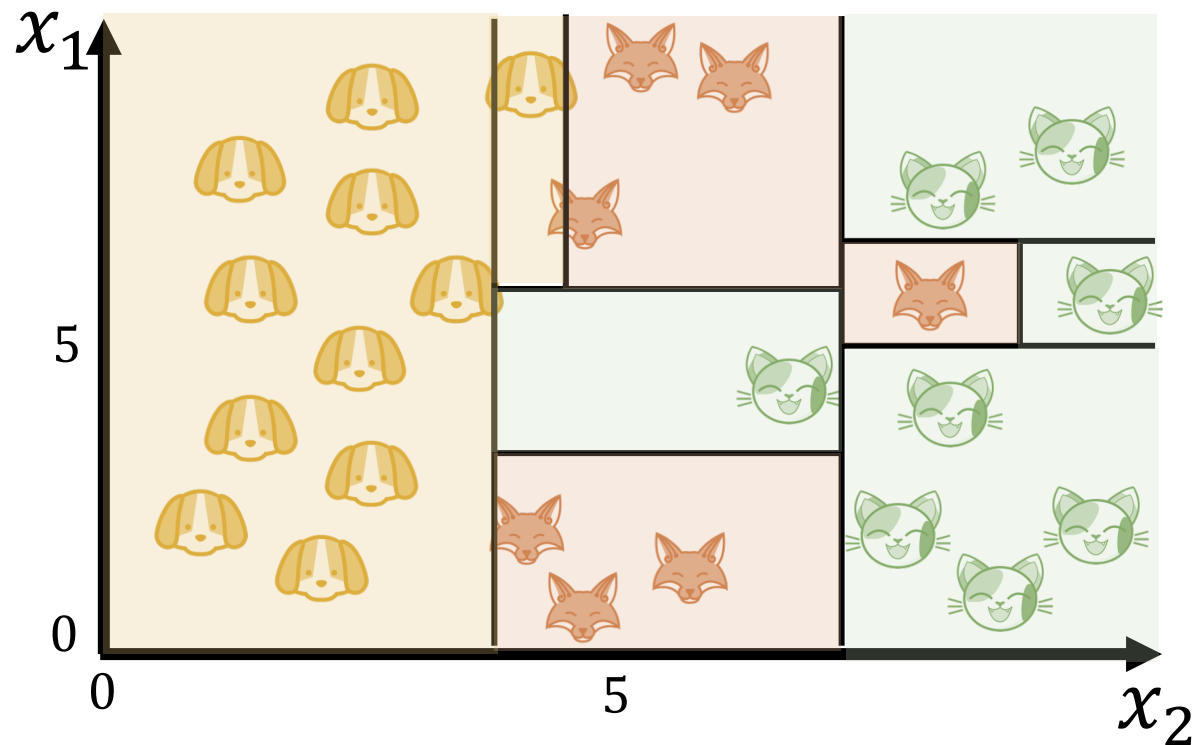
- **Continuous:** Cylinder Volume, Vehicle Weight
- **Discrete:** Manufacturer, Diesel/Regular

Don't really need to do anything special for this case.

- At each step consider all possible splits including:
 - Splits on discrete attributes
 - Threshold splits in-between datapoints for continuous attributes



As decision trees get bigger (deeper), they can represent very complex functions.



Really large trees can achieve minimal error on training sets, but may not generalize to test data - overfitting.



Really large trees can achieve minimal error on **training sets**, but may not generalize to **test data** – overfitting.





Option 1: Add more hyperparameters to control tree size:

- Limit depth / limit number of nodes / only split if at least K datapoints present

Option 2: Early stopping based on validation performance

- Monitor the validation accuracy and stop splitting a branch when performance saturates.
- Can be tricky for the same reasons that our proposed Base Case 3 can be.

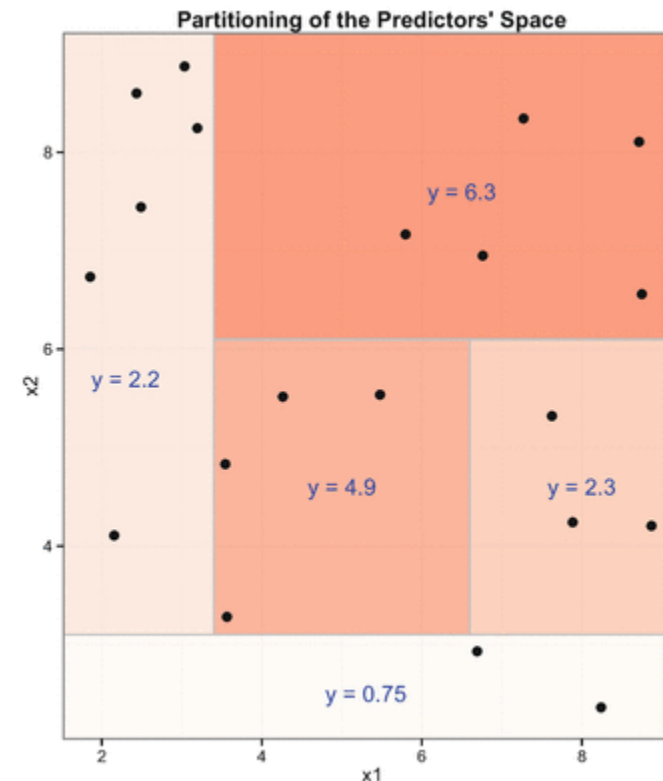
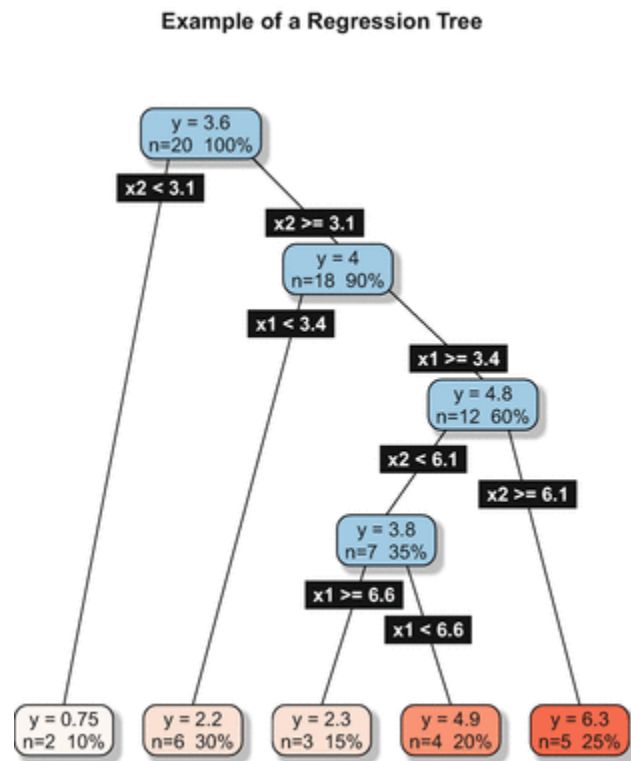
Option 3: Post Pruning

- Grow full tree on the training set, then consider the impact of removing each node on validation performance.
- Greedily prune the node that most improves validation set performance.



We've talked mostly about classification trees. Can also do regression:

- Leaf nodes store the average output value of datapoints in them rather than the majority class.
- Rather than conditional entropy, consider something like change in squared error when choosing splits.



Today's Learning Objectives

Be able to answer:

- ~~What are the intuitions for some advanced structures in neural networks?~~

—

- ~~What are decision trees?~~

- ~~How are they learned?~~

- ~~What is entropy, conditional entropy, and information gain?~~

- ~~How do they deal with continuous features?~~

—

- ~~What do to about overfitting?~~





Next Time: We'll keep talking a bit about Decision Trees and ensemble methods.